
Is there a way to improve language models accuracy in solving physics problems in Russian and English languages?

Sergei Mironichev

High school student from school №1329

Igor Obolensky

Physics teacher from school №1329

Abstract

Large language models are increasingly used to solve educational physics problems, but their accuracy remains inconsistent. This study investigates whether prompt formulation can improve the performance of language models on school-level physics problems in Russian and English. In total 21 language models were tested on 25 problems with 3 different methods: original textbook problem statements, simplified statements written in a structured Given/Find format, and original statements supplemented with solved examples from the same area of physics. Answers were evaluated using numerical results, formulas, and physical units, with numerical accuracy used as the primary performance measure. The original problem statements achieved the highest overall numerical accuracy at 38.7%. Simplifying the statements gave 26.3% accuracy, while adding solved examples resulted in 34.5% accuracy. English-source problems showed higher numerical accuracy than Russian-source problems in all three conditions, but the datasets differed in difficulty, preventing this difference from being attributed to language alone. The results suggest that additional prompt structure or context does not necessarily improve physics problem solving by language models and that preserving information contained in natural textbook-style problem statements may be important. Scripts and data for this experiment are available in https://github.com/BOCK134/Physics_problems_solving_methods.

Table of Contents

1	Introduction	2
2	Collecting data.....	2
3	Getting data ready	6

4 The first phase of experiment	7
5 The second phase of experiment.....	12
5.1 First method	12
5.2 Second method	14
6 Data and analysis	18
6.1 Overall score	18
6.2 Language difference.....	22
6.3 Cost	23
7 Conclusion and evaluation	26
8 Limitations and improvements.....	27
9 Bibliography.....	<i>Error! Bookmark not defined.</i>
10 Addendum	29
9.1 Russian original statements	29
9.2 First script:	30
9.3 Second script	41

1 Introduction

Language models are now often used for educational tasks, including solving school-level physics problems. However, solving a physics problem is not only a text-generation task. The model must understand the condition, identify the relevant physical law, substitute the correct values, calculate the numerical answer, and give the correct units.

When AI tries to solve physics problems the answers are not always accurate which made me wonder why language models cannot always correctly solve school level problems.

That's why I took the opportunity to answer a question: 'is there a way to improve AI model solving accuracy in English and Russian physics problem solving?'

2 Collecting data

I needed to find a trustworthy source to collect data (problems) from. My supervisor recommended a Russian student book called "3,800 Problems for School Students and

University Applicants”¹. Problems in Russian language were chosen from here. For the experiment I wanted at least another English problem source so as a second source I chose OpenStax.

The problems were selected from the main areas of school physics: mechanics, gas laws and thermodynamics, optics, electrostatics, and nuclear physics. It was an essential choice because it tests most of the aspects of physics. Each area had 5 problems. In total there were 25 problems. Some chosen problems were marked as high difficulty by the source which are also included. Problems with pictures also weren’t included in the list because of the difficulty of sending it to the AI.

Table 1 contains English and Russian problems with Russian statements translated to English. The original Russian problem statements can be found in 9.1 Addendum section.

Table 1. The chosen physics problems for the experiment

№	Statement (source)	Ans formula LaTeX ans number (ans units)
mechanics		
1	A particle travels a distance $l = 2$ m with constant velocity, and then slows down with acceleration $a = 5 \cdot 10^5$ m/s ² . At what speed of the particle will the time of motion from start to stop will be the smallest? (3,800 Problems for School Students and University Applicants)	$\sqrt{a l}$ 1000 (m/s)
2	A point moves along a straight line according to the law $x = At + Bt^3$, where $A = 6$ m/s; $B = -0,125$ m/s ³ . Determine: the average speed of the point in the time interval from $t_1 = 2$ s to $t_2 = 6$ s; (3,800 Problems for School Students and University Applicants)	$A + B \left(t_1^2 + t_1 t_2 + t_2^2 \right)$ -0.5 (m/s)
3	A truck is traveling east at 80 km/h. At an intersection 32 km ahead, a car is traveling north at 50 km/h. How long in hours after this moment will the vehicles be closest to each other? (https://openstax.org/books/university-physics-volume-1/pages/preface)	$\frac{d v_1}{\sqrt{v_1^2 + v_2^2}}$ 0.29 (h)
4	The law of motion of a material point has the form: $x = a_{\{1\}} + b_{\{1\}}t + y_{\{1\}}t^2$; $y = a_{\{2\}} + b_{\{2\}}t + y_{\{2\}}t^2$, где $b_{\{1\}} = b_{\{2\}} = 1$ м/с, $y_{\{1\}} = -1$ м/с ² , $y_{\{2\}} = 2$ м/с ² . What is its acceleration? Determine the angle between the speed and the acceleration at the time $t = 1$ s. (3,800 Problems for School Students and University Applicants)	$\arccos\left\{\frac{a_x v_x + a_y v_y}{a v}\right\}$ - (degrees)
5	A crate is being pushed across a rough floor surface. If no force is applied on the crate, the crate will slow down and come to a stop. If the crate of mass 50 kg moving at speed 8 m/s comes to rest in 10 seconds, what is the rate at which the frictional force on the crate takes energy away from the crate? (https://openstax.org/books/university-physics-volume-1/pages/7-additional-problems)	$\frac{m \left(v_0^2 - v_f^2 \right)}{2 \Delta t}$ 160 (J/s)

¹ Original name “3800 задач по физике для школьников и поступающих в вузы”

Nº	Statement (source)	Ans formula LaTeX ans number (ans units)
gas laws and thermodynamics		
6	Eight bumper cars, each with a mass of 322 kg, are running in a room 21.0 m long and 13.0 m wide. They have no drivers, so they just bounce around on their own. The rms speed of the cars is 2.50 m/s. Repeating the arguments of Pressure, Temperature, and RMS Speed, find the average force per unit length (analogous to pressure) that the cars exert on the walls. (https://openstax.org/books/university-physics-volume-2/pages/2-challenge-problems)	$\frac{N m v_{rms}^2}{2 W l}$ 29.5 (N/m)
7	What work (in kJ) is made during isothermal expansion of hydrogen, taken at a temperature of $t = 11^\circ\text{C}$, if its volume increases by 3 times? The mass of hydrogen is $m = 5\text{ g}$. (3,800 Problems for School Students and University Applicants)	$\frac{R T m}{\ln\left(\frac{V_2}{V_1}\right)}$ 6.5 (kJ)
8	When an air bubble rises from the bottom to the top of a freshwater lake, its volume increases by 80%. If the temperatures at the bottom and the top of the lake are 4.0°C and 10°C , respectively, how deep is the lake? (https://openstax.org/books/university-physics-volume-2/pages/2-additional-problems)	$\frac{p_0 \left(\frac{T_1 V_2}{T_2 V_1} - 1\right)}{g \rho}$ 7.9 (m)
9	On a warm day when the air temperature is 30°C , a metal can is slowly cooled by adding bits of ice to liquid water in it. Condensation first appears when the can reaches 15°C . What is the relative humidity of the air? (https://openstax.org/books/university-physics-volume-2/pages/2-additional-problems)	$10^{\left\{-\frac{7.5 T}{T + 237.3} + \frac{7.5 T_d}{T_d + 237.3}\right\}}$ 100% 40.18 (%)
10	A gas occupying a volume $V_1 = 10\text{ L}$ at a pressure $p_1 = 2 \times 10^5\text{ Pa}$ expands isothermally to a volume $V_2 = 28\text{ L}$. What work (in kJ) does it perform when doing so? (3,800 Problems for School Students and University Applicants)	$V_1 p_1 \ln\left(\frac{V_2}{V_1}\right)$ 2.06 (kJ)
optics		
11	The print in many books averages 3.50 mm in height. How high in mm is the image of the print on the retina when the book is held 30.0 cm from the eye? (https://openstax.org/books/university-physics-volume-3/pages/2-problems)	$-h_o \left(\frac{d_i}{d_o}\right)$ -0.23 (mm)
12	What is the angular magnification of a telescope that has a 100 cm-focal length objective and a 2.50 cm-focal length eyepiece? (https://openstax.org/books/university-physics-volume-3/pages/2-problems)	$\frac{f_{\text{obj}}}{f_{\text{eye}}}$ -40 (1)
13	A ray is consequentially reflected from two flat mirrors, while the ray dropping on the first mirror is parallel to the plane of the second mirror, and the twice-reflected ray is parallel to the plane of the first mirror. What is the angle ϕ between the mirrors? (3,800 Problems for School Students and University Applicants)	- 60 (degrees)
14	On the vessel filled with a water likes a flat mirror. A person leaning over the vessel sees the image of his face in the mirror at the distance $d = 25\text{ cm}$, if the distance from the face to the water surface is $h = 5\text{ cm}$. Determine the depth of the vessel l in cm. (3,800 Problems for School Students and University Applicants)	$n \left(\frac{d}{2} - h\right)$ 10 (cm)

Nº	Statement (source)	Ans formula LaTeX ans number (ans units)
15	Calculate the wavelength of light in nm that produces its first minimum at an angle of 36.9° when falling on a single slit of width 1.00 μm. (https://openstax.org/books/university-physics-volume-3/pages/4-additional-problems)	$a \sin(\theta)$ 600 (nm)
electrostatics		
16	At the ends of a thin non-conducting horizontal rod with length $l = 1$ m, two small beads are fixed, and a third is threaded onto the rod, along this rod it can move without friction. All the beads are given equal charges $q = 10^{-6}$ C. Find the period of small oscillations of the movable bead. The mass of the bead is $m = 5$ g. (3,800 Problems for School Students and University Applicants)	$\frac{\pi l q}{\epsilon_0 m \omega^2}$ 0.83 (s)
17	At an electric field with strength of $E = 10^6$ V/m, air stops to be a reliable insulator and a spark discharge occurs in it. What must be the radius of a metal sphere so that a charge $q = 1$ C can be held on it? (3,800 Problems for School Students and University Applicants)	$\frac{1}{2} \sqrt{\frac{q \pi}{\epsilon_0 E}}$ 94.9 (m)
18	An electron moves in the direction of a field line of a uniform electric field, whose strength is $E = 100$ V/m. What distance will it travel before a complete stop in cm, if the electron's initial speed is $v = 10^6$ m/s? (3,800 Problems for School Students and University Applicants)	$\frac{m_e v^2}{2 e E}$ 2.8 (cm)
19	Two identical air capacitors are connected in series and connected to a battery with constant EMF. One of them is filled with a dielectric with dielectric permittivity $\epsilon = 4$. By what factor the electric field strength in this capacitor will change? (3,800 Problems for School Students and University Applicants)	$\frac{2}{\epsilon + 1}$ 0.4 (1)
20	A particle of mass m and charge $-q$ moves along a straight line away from a fixed particle of charge Q . When the distance between the two particles is r_0 , $-q$ is moving with a speed v_0 . What do you have to assume about v_0 to make this calculation? (formula only) (https://openstax.org/books/university-physics-volume-2/pages/5-problems)	$\sqrt{\frac{Q q}{2 \pi \epsilon_0 m r_0}}$ - (-)
nuclear		
21	During transition of electrons from some more distant orbit to the second Bohr orbit, hydrogen atoms emit a monochromatic beam of light. When this beam is falls on normal line on a diffraction grating, a second-order maximum ($k = 2$) is observed at a diffraction angle $\phi = 30^\circ$. The diffraction grating constant is $d = 2.6$ μm. Determine the number n of the orbit on which the electrons in the hydrogen atoms were originally located. (3,800 Problems for School Students and University Applicants)	$2 \sqrt{\frac{d \cos \phi}{\lambda}}$ 5 (1)
22	The submarine "Nautilus" (USA) has a fuel plant power of $P = 14,7$ MW, with an efficiency of $\eta = 25\%$. The fuel is enriched uranium with a mass of $m_0 = 1$ kg, and the fission of whose nuclei releases energy $E = 6,9 \cdot 10^{13}$ J. Determine the fuel reserve required for a year long work of the submarine. (3,800 Problems for School Students and University Applicants)	$m_0 \frac{P}{\eta E}$ 26.9 (kg)

Nº	Statement (source)	Ans formula LaTeX ans number (ans units)
23	To determine the half-life of a short-lived radioactive isotope a pulse counter was used. In first minute at the start of observation, $\Delta n_0 = 250$ pulses were counted, and after a time $t = 1$ h, $\Delta n = 92$ pulses were registered. What is the half-life in min of this isotope? (3,800 Problems for School Students and University Applicants)	$t \frac{\lg 2}{\lg \frac{\Delta n_0}{\Delta n}}$ 42 (min)
24	The wiki-phony site states that the atomic mass of chlorine is 40 g/mol. Check this result. Hint: The two, most common stable isotopes of chlorine are: ${}^{35}_{17}\mathrm{Cl}$ and ${}^{37}_{17}\mathrm{Cl}$. (The abundance of Cl-35 is 75.8%, and the abundance of Cl-37 is 24.2%). (https://openstax.org/books/university-physics-volume-3/pages/10-problems)	$f_{35} M_{35} + f_{37} M_{37}$ 35.5 (g/mol)
25	In the 1980s, the term picowave was used to describe food irradiation in order to overcome public resistance by playing on the well-known safety of microwave radiation. Find the energy in MeV of a photon having a wavelength of a picometer. (https://openstax.org/books/university-physics-volume-3/pages/10-problems)	$\frac{hc}{\lambda}$ 1.24 (MeV)

3 Getting data ready

Problems were chosen but they needed some formatting as well as their answers. In total the AI model could get up to 3 points: 1 for valid numeric answer, 1 for valid formula answer and the last one for writing correct Physics units. It only counts if it is the same. At first some problems had to be shortened: if the original problem had several subsections, one was often left for the experiment, so that each line had one testable objective. The formulas mostly were given in the sources but needed to be rewritten in a single format. Final actual formulas were rewritten and the statement corrected: all variables in the statement had to be in the LaTeX form including special symbols (for example π as $\backslash\pi$). For the unit scoring the clarification at the end of the statement was added that mentions in what units the answer must be. However, it was not applied to every statement. Problems that had units in the SI system are affected by this.

The prompt defines in what format language model's answer should be:

The answer that the AI model should give is a JSON file that has fields: "final_numeric", "final_formula" and "final_units". The prompt states the format of output: no extra text just JSON and clarifies some nuances. If there is no numeric answer (in case if only formula is needed) it should state it as null. Same goes for units: if the answer is unitless, model should return 1. The field "final_formula" should contain an end math expression in LaTeX form only. Prompt has a rule that all units must be in the SI system until there is an exact unit requested by statement.

Prompt:

You are an expert in physics problems who solves physics problem.

INPUT

The following is a physics problem that needs to be solved.

{{PROBLEM}} - pastes there statement

OUTPUT REQUIREMENTS

Answer should be only in .JSON format (no markdown, no extra text).

Return a single JSON object with these required keys:

- "final_numeric": the final numeric value as a JSON number (not a string). If the answer is symbolic-only, use null.
- "final_formula": a string containing the final answer as a formula/expression written in LaTeX rules.
- "final_units": a string with SI units for the numeric answer (e.g., "N", "J", "V/m", "C", "kg*m/s^2"). If unitless, use "1". If numeric answer is null, still provide the expected units for the expression. Use C system if it is not stated in what units answer needs to be. If unitless, use "1".

OUTPUT FORMAT

```
{  
  "final_numeric": 1,  
  "final_formula": "formula",  
  "final_units": "m",  
}
```

4 The first phase of experiment

Firstly, the models had to be chosen. I had to choose different types of AI models for more interesting results. To search for these, I used OpenRouter which will be used later to send problems.

Table 2. Selected AI models

Nº	date tested 1st phase	model id	Model Name	Country	Model Year
1	19 Apr 2026	x-ai/grok-4.20	xAI: Grok 4.20	USA	Q3 2025
2	19 Apr 2026	openai/gpt-5.4	OpenAI: GPT-5.4	USA	Q1 2026
3	19 Apr 2026	moonshotai/kimi-k2.5	MoonshotAI: Kimi K2.5	China	Q1 2026

Nº	date tested 1st phase	model id	Model Name	Country	Model Year
4	19 Apr 2026	anthropic/claude-sonnet-4.5	Anthropic: Claude Sonnet 4.5	USA	Q3 2025
5	19 Apr 2026	google/gemma-4-31b-it	Google: Gemma 4 31B	USA	Q1 2026
6	19 Apr 2026	google/gemini-2.5-pro	Google: Gemini 2.5 Pro	USA	Q2 2025
7	19 Apr 2026	anthropic/claude-opus-4.7	Anthropic: Claude Opus 4.7	USA	Q1 2026
8	19 Apr 2026	qwen/qwen3.5-397b-a17b	Qwen: Qwen3.5 397B A17B	China	Q1 2026
9	19 Apr 2026	deepseek/deepseek-r1	DeepSeek: R1	China	Q1 2025
11	22 Apr 2026	qwen/qwen3.6-plus	Qwen: Qwen3.6 Plus	China	Q2 2026
12	22 Apr 2026	openai/gpt-5.4-pro	OpenAI: GPT-5.4 Pro	USA	Q1 2026
14	22 Apr 2026	mistralai/ministral-14b-2512	Mistral: Ministral 3 14B 2512	France	Q1 2025
15	22 Apr 2026	google/gemini-3.1-pro-preview	Google: Gemini 3.1 Pro Preview	USA	Q1 2026
16	22 Apr 2026	xiaomi/mimo-v2.5-pro	Xiaomi: MiMo-V2.5-Pro	China	Q2 2026
17	22 Apr 2026	moonshotai/kimi-k2.6	MoonshotAI: Kimi K2.6	China	Q1 2026
18	22 Apr 2026	openai/o4-mini-high	OpenAI: o4 Mini High	USA	Q2 2025
19	22 Apr 2026	stepfun/step-3.5-flash	StepFun: Step 3.5 Flash	China	Q1 2026
20	22 Apr 2026	meta-llama/llama-4-maverick	Meta: Llama 4 Maverick	USA	Q2 2025
21	22 Apr 2026	openai/gpt-oss-120b	OpenAI: gpt-oss-120b	USA	Q3 2025
22	22 Apr 2026	nvidia/llama-3.3-nemotron-super-49b-v1.5	NVIDIA: Llama 3.3 Nemotron Super 49B V1.5	USA	Q4 2025
23	22 Apr 2026	microsoft/phi-4	Microsoft: Phi 4	USA	Q1 2025

Two of the models (*pixtral-large-2411* and *deepseek-v3.2-speciale*) are excluded from the experiment. These models were tested in the first phase, but when it was time for the second phase, they were not available anymore.

Two python scripts were written for this experiment.

The first script gets a .CSV file with problems' statements and adding prompts to it one by one sends them to the AI model from the list. In the end the script gets 3 files: JSON, csv and log. Log is for debugging. JSON is the answer that is requested from the model. CSV contains several problems, all answers in different columns, raw answers and in, out tokens.

At first the number of problems that should be sent and from which is defined.

```
PROBLEM_MAX = 25
PROBLEM_START = 1
```

This is made for an opportunity to send exact problems to the AI model if needed in case of failing to give answer for a specific problem.

Next AI model is defined:

```
AI_MODELS = [  
    "anthropic/claude-opus-4.7" #to test claude opus 4.7  
]
```

Other options of language model are also identified:

```
AI_MODEL_DEFAULT_TEMPERATURE = 0.1  
API_URL = "https://openrouter.ai/api/v1/chat/completions"  
  
REQUEST_TIMEOUT = 1800  
MAX_RETRIES = 2  
SLEEP_BETWEEN_REQUESTS_SEC = 1.0  
REQUIRED_OUTPUT_FIELDS = ["final_numeric", "final_formula", "final_units"]
```

A low temperature of 0.1 is used to make model answers more stable and reproducible. The long timeout is necessary because reasoning models can take several minutes to answer. The script can also repeat unsuccessful requests and waits one second between model calls.

Then script paste the problem's statement replacing {{PROBLEM}} in the prompt.

```
def build_user_prompt(problem_text):  
    return PROMPT_TEMPLATES[PROMPT_VERSION].replace("{{PROBLEM}}",  
    problem_text)
```

```
for i, row in enumerate(reader):  
    # check if have statement is not empty  
    statement = row.get(statement_field)  
  
    if statement is None:  
        continue  
  
    # also skip if not reached start problem yet  
    if i < start_problem-1:  
        continue  
  
    statement = str(statement).strip()  
    if not statement:  
        continue  
  
    items.append({  
        # add +1 because have first row is HEADER (1 row)  
        "statement_row_id": i+1,  
        "statement": statement  
    })  
  
    if len(items) >= max_items:  
        break
```

This part searches for the Statement column in the CSV file and reads problems from it. Empty rows are ignored. Rows before PROBLEM_START are skipped, and loading stops when the number specified by PROBLEM_MAX has been reached.

Each loaded problem is stored together with its row number so that the model response can later be connected to the original problem.

The function `call_ai` creates a payload containing the model's name, temperature, and completed prompt. It then sends this payload to the OpenRouter API.

```
for idx, item in enumerate(problems, start=1):
    statement_row_id = item["statement_row_id"]
    statement = item["statement"]
    print(f"[{idx}/{len(problems)}] Sending problem #{statement_row_id}
(csv row {statement_row_id+1})..")

    for ai_model in AI_MODELS:
        api_result = call_ai(
            ai_model, # ai model
            statement_row_id, # problem id from CSV file (first column)
            statement, # problem statement
            OPENROUTER_API_KEY
        )
```

This is the main experimental loop. The outer loop goes through the selected problems, while the inner loop sends each problem to every active model.

Then, as an language model output script return these fields:

```
out = {
    "ok": False,
    "datetime": formatted_datetime,
    "ai_model": ai_model,
    "statement_row_id": statement_row_id,
    "statement": statement,
    "final_numeric": None,
    "final_formula": None,
    "final_units": None,
    "error": api_result.get("error"),
    "raw_response_text": api_result.get("raw_response_text"),
    "full_api_response": api_result.get("full_api_response"),
    "prompt_tokens": api_result.get("prompt_tokens"),
    "completion_tokens": api_result.get("completion_tokens"),
    "total_tokens": api_result.get("total_tokens"),
    "prompt_version": PROMPT_VERSION
}
```

The script records not only the final answer but also the original problem, model name, errors, complete raw response, token usage, and prompt version. These fields are useful for checking parsing errors, calculating costs, and comparing different models or prompt variants.

Lastly, results are downloaded:

```
files.download(file_csv)
files.download(file_jsonl)
```

During the first tests script had several issues where it didn't save the results. Some error codes were added as a new column in csv and results were saved even if there is an error. There was also a problem when the AI model thought for a long time and Google Collaborate disconnected while it was waiting for a long time for a response which led to a longer request timeout. in figure 1 you can see the finish reason is length.

May 27, 9:12 PM		May 28, 2:12 AM		May 28, 7:12 AM		May 28, 12:12 PM			
Date	Model	Provider	App 	Input	Output	Cost	Usage Type	Speed	Finish Reason
May 28, 09:10 PM	 R1	 Azure	Unknown	556 tok	4,096 tok	\$ 0.0252	Credits	78.2 tok/s	length

Fig. 1. Model stopped unexpectedly because of the length

The second script merges all the results into one CSV file and from the column raw_response took cost information from each operation. In the end there was one big file with all the results that needed to be analyzed.

Results of all language models for the original statement problems set are showed in the Table 3.

Total numeric - amount of accurate given numeric answers

Total formula - amount of accurate given formula answers

Total units - amount of accurate given unit answers

Total score - sum of total units, total numeric and total formula

Table 3. Results for original problems set

Model	Total numeric	Total formula	Total units	Total score	Total Tokens input	Total tokens output	Tokens total	Total cost (\$)
anthropic/claude-opus-4.7	12	1	19	32	20 995	4 567	25 562	0.3
x-ai/grok-4.20	3	1	19	23	15 560	1 024	16 584	0.1
openai/gpt-5.4	6	0	14	20	12 848	2 115	14 963	0.1
moonshotai/kimi-k2.5	13	1	18	32	13 328	131 205	144 533	0.4
anthropic/claude-sonnet-4.5	11	0	19	30	14 919	11 316	26 235	0.3
google/gemma-4-31b-it	3	1	20	24	14 396	1 435	15 831	0.1
google/gemini-2.5-pro	10	2	16	28	14 099	111 582	125 681	1.2
qwen/qwen3.5-397b-a17b	12	1	16	29	13 646	160 387	174 033	0.5
deepseek/deepseek-r1	14	1	19	34	13 761	110 147	123 908	0.4

Model	Total numeric	Total formula	Total units	Total score	Total Tokens input	Total tokens output	Tokens total	Total cost (\$)
mistralai/pixtral-large-2411	3	1	17	21	16 525	1 616	18 141	0.1
qwen/qwen3.6-plus	15	0	21	36	13 646	103 298	116 944	0.3
openai/gpt-5.4-pro	11	1	18	30	12 848	31 257	44 105	6.3
deepseek/deepseek-v3.2-speciale	12	1	19	32	13 632	214 295	227 927	0.3
mistralai/ministral-14b-2512	3	0	14	17	13 627	2 079	15 706	0.1
google/gemini-3.1-pro-preview	11	1	21	33	14 096	78 611	92 707	1
xiaomi/mimo-v2.5-pro	12	2	17	31	19 004	134 093	153 097	0.5
moonshotai/kimi-k2.6	10	2	16	28	13 265	204 507	217 772	0.9
openai/o4-mini-high	12	2	17	31	12 848	91 203	104 051	0.5
stepfun/step-3.5-flash	10	0	17	27	13 037	490 326	503 363	0.2
meta-llama/llama-4-maverick	7	0	17	24	12 993	1 128	14 121	0.1
openai/gpt-oss-120b	13	0	17	30	14 251	25 444	39 695	0.1
nvidia/llama-3.3-nemotron-super-49b-v1.5	11	1	19	31	13 037	113 261	126 298	0.1
microsoft/phi-4	4	1	15	20	13 368	1 813	15 181	0.1

The first phase showed that Qwen 3.6 Plus and DeepSeek R1 achieved the strongest numerical performance, although no language model answered more than 60% of the problems correctly which was surprising for this set of school problems. The first-phase results were used mainly as a baseline for selecting difficult problems and testing improved task formulations.

5 The second phase of experiment

The second phase is created to check if there are any methods for score improvement by changing the prompt. The next step was to choose what methods to use to theoretically improve results. Previous research has investigated structured physics reasoning, few-shot prompting, and the use of worked examples in scientific problem solving. [1] [2] [3] Based on these approaches, I decided to test two methods: restructuring the problem statement and adding solved examples.

5.1 First method

The first method is to simplify the statement to the point where there are only given variables values, some notes about the statement. For example:

Table 4. Example of simplified statement vs original

Nº	Original	Simplified
3	A truck is traveling east at 80 km/h. At an intersection 32 km ahead, a car is traveling north at 50 km/h. How long in hours after this moment will the vehicles be closest to each other?	Given: $v_{\{1\}} = 80 \text{ km/h}$ — east truck's speed $v_{\{2\}} = 50 \text{ km/h}$ — north car's speed $d = 32 \text{ km}$ — distance from the truck to the intersection at the initial moment. Find: t — the time in hours after this moment when the vehicles will be closest to each other.

All statements were simplified to the same format: Given: ... Find: ...

The purpose of this transformation was not to make the calculations easier, but to reduce ambiguity in the interpretation of the problem and make the relevant physical quantities more explicit. [1] [4] The same physical situation, numerical values, and requested quantity were preserved. Only the way in which the information was presented was changed.

Before testing this method, each rewritten statement was compared with its original version to ensure that its physical meaning had not changed. While comparing some mistakes in the statements were found. With corrected and reviewed statements there were 2 problems (Nº 2 & 23) to be sent again in the first phase then to replace the old incorrect ones.

The hypothesis was that a structured Given-Find format would reduce mistakes caused by misunderstanding the text, overlooking a condition, or confusing the required units. If the numerical accuracy improved, this would indicate that interpretation of the original language was one of the difficulties. If the results remained similar, the main limitation would more likely be the mathematical or physical reasoning itself.

Below the results for the simplified statements set are provided.

Table 5. Results for simplified problems set

Model	Total numeric	Total formula	Total units	Total score	Total Tokens input	Total tokens output	Tokens total	Total cost (\$)
anthropic/claude-opus-4.7	9	0	19	28	21 116	4 536	25 652	0.3
x-ai/grok-4.20	3	1	19	23	15 643	1 004	16 647	0.1
openai/gpt-5.4	3	0	14	17	12 916	1 919	14 835	0.1
moonshotai/kimi-k2.5	9	0	20	29	13 383	137 482	150 865	0.4
anthropic/claude-sonnet-4.5	8	0	13	21	15 222	12 492	27 714	0.3

Model	Total numeric	Total formula	Total units	Total score	Total Tokens input	Total tokens output	Tokens total	Total cost (\$)
google/gemma-4-31b-it	4	0	20	24	14 663	1 436	16 099	0.1
google/gemini-2.5-pro	9	0	14	23	14 363	103 777	118 140	1.1
qwen/qwen3.5-397b-a17b	7	1	20	28	13 942	132 998	146 940	0.5
deepseek/deepseek-r1	8	1	17	26	13 740	100 805	114 545	0.4
qwen/qwen3.6-plus	9	2	20	31	13 942	97 793	111 735	0.2
openai/gpt-5.4-pro	8	0	18	26	12 916	27 113	40 029	5.3
mistralai/ministral-14b-2512	1	0	15	16	13 682	1 694	15 376	0.1
google/gemini-3.1-pro-preview	8	1	21	30	14 352	64 722	79 074	0.9
xiaomi/mimo-v2.5-pro	8	0	19	27	19 116	168 610	187 726	0.2
moonshotai/kimi-k2.6	7	1	17	25	13 347	182 982	196 329	0.8
openai/o4-mini-high	10	1	19	30	12 916	64 763	77 679	0.3
stepfun/step-3.5-flash	5	1	17	23	13 717	452 794	432 955	0.2
meta-llama/llama-4-maverick	6	0	16	22	12 828	1 148	13 976	0.1
openai/gpt-oss-120b	8	0	16	24	14 343	27 028	41 371	0.1
nvidia/llama-3.3-nemotron-super-49b-v1.5	7	0	18	25	13 129	137 726	150 855	0.1
microsoft/phi-4	1	0	14	15	14 829	1 275	16 104	0.1

5.2 Second method

The second method's idea was to add examples for each category to all problems in that category.

The original problem statement was kept unchanged, but three solved problems from the same area of physics were added below it. The purpose was to show the model relevant solution patterns before it attempted the target problem. This approach is related to few-shot prompting, where examples are included in the prompt to demonstrate the expected reasoning or solution pattern before the answers a new problem. [2] [3]

The problems have been chosen from OpenStax (same source for English problems).

For example, to the first five problems these examples were added after the statement:

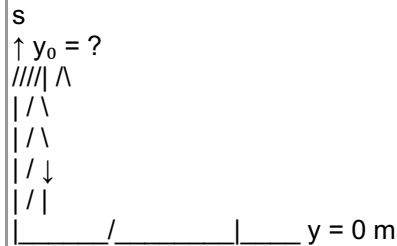
{{STATEMENT}}

Examples:

Example 1:

Calculate the height of a cliff if it takes 2.35 s for a rock to hit the ground when it is thrown straight up from the cliff with an initial velocity of 8.00 m/s. (b) How long would it take to reach the ground if it is thrown straight down with the same speed?*

Solution



(a) Knowns: $t = 2.35 \text{ s}$; $y = 0 \text{ m}$; $v_0 = +8.00 \text{ m/s}$; $a = -9.8 \text{ m/s}^2$

Since we know t , y , v_0 , and a and want to find y_0 , we can use the equation

$$y = y_0 + v_0 t + \frac{1}{2} a t^2$$

$$y = (0 \text{ m}) + (+8.00 \text{ m/s})(2.35 \text{ s}) + \frac{1}{2}(-9.80 \text{ m/s}^2)(2.35 \text{ s})^2 = -8.26 \text{ m}$$

so the cliff is

8.26 m high.

(b) Knowns: $y = 0 \text{ m}$; $y_0 = 8.26 \text{ m}$; $v_0 = -8.00 \text{ m/s}$; $a = -9.80 \text{ m/s}^2$

Now we know y , y_0 , v_0 , and a and want to find t , so we use the equation

$$y = y_0 + v_0 t + \frac{1}{2} a t^2$$

again. Rearranging,

$$t = \frac{-v_0 \pm \sqrt{v_0^2 - 4(0.5a)(y_0 - y)}}{2(0.5a)}$$

$$t = \frac{-(-8.00 \text{ m/s}) \pm \sqrt{(-8.00 \text{ m/s})^2 + 2(9.80 \text{ m/s}^2)(8.26 \text{ m} - 0 \text{ m})}}{-9.80 \text{ m/s}^2}$$

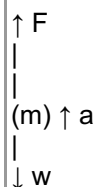
$$= \frac{8.00 \text{ m/s} \pm 15.03 \text{ m/s}}{-9.80 \text{ m/s}^2}$$

$$t = 0.717 \text{ s} \text{ or } -2.35 \text{ s} \rightarrow t = \underline{0.717 \text{ s}}$$

Example 2:

Calculate the force a 70.0-kg high jumper must exert on the ground to produce an upward acceleration 4.00 times the acceleration due to gravity. Explicitly show how you follow the steps in the Problem-Solving Strategy for Newton's laws of motion.

Solution Step 1. Use Newton's Laws of Motion.



Force acting on
the high jumper.

Step 2. Given: $a = 4.00g = (4.00)(9.80 \text{ m/s}^2) = 39.2 \text{ m/s}^2$; $m = 70.0 \text{ kg}$

Find F .

Step 3. $\sum F = +F - w = ma$, so that $F = ma + w = ma + mg = m(a + g)$

$$F = (70.0 \text{ kg})[(39.2 \text{ m/s}^2) + (9.80 \text{ m/s}^2)] = \underline{3.43 \times 10^3 \text{ N}}$$

The force exerted by the high-jumper is actually down on the ground, but F is up from the ground to help him jump.

Step 4. This result is reasonable, since it is quite possible for a person to exert a force of the magnitude of (10^3 N) .

Example 3:

A shopper pushes a grocery cart 20.0 m at constant speed on level ground, against a 35.0 N frictional force. He pushes in a direction 25.0° below the horizontal. (a) What is the work done on the cart by friction? (b) What is the work done on the cart by the gravitational force? (c) What is the work done on the cart by the shopper? (d) Find the force the shopper exerts, using energy considerations. (e) What is the total work done on the cart?

Solution (a) The work done by friction is in the opposite direction of the motion, so $(\theta = 180^\circ)$,

$$\text{and therefore } W_f = Fd \cos \theta = 35.0 \text{ N} \times 20.0 \text{ m} \times \cos 180^\circ = \underline{-700 \text{ J}}$$

(b) The work done by gravity is perpendicular to the direction of motion, so $(\theta = 90^\circ)$,

$$\text{and } W_g = Fd \cos \theta = 35.0 \text{ N} \times 20.0 \text{ m} \times \cos 90^\circ = \underline{0 \text{ J}}$$

(c) If the cart moves at a constant speed, no energy is transferred to it, from the work-energy theorem: $(W_{\text{net}} = W_s + W_f = 0)$ or $(W_s = \underline{700 \text{ J}})$

(d) Use the equation $(W_s = Fd \cos\theta)$, where $(\theta = 25^\circ)$, and solve for the force:

$$\begin{aligned} F &= \frac{W_s}{d \cos\theta} \\ &= \frac{700 \text{ J}}{20.0 \text{ m} \times \cos 25^\circ} \\ &= 38.62 \text{ N} \\ &= \underline{38.6 \text{ N}} \end{aligned}$$

(e) Since there is no change in speed, the work energy theorem says that there is no net work done on the cart: $(W_{\text{net}} = W_f + W_s = -700 \text{ J} + 700 \text{ J} = \underline{0 \text{ J}})$

Some examples as can be seen in example contained diagrams that were part of the problem statement and included information not fully repeated in the text. Therefore, these images were recreated as LaTeX schemes. Labels, arrows, directions, angles, distances, and the relative positions of objects were preserved. This allowed the models to interpret the visual information through its structured LaTeX representation when solving the problem. Each reconstructed scheme was checked against the original image to ensure that no relevant physical information was omitted or changed.

Table 6. Results with examples

Model	Total numeric	Total formula	Total units	Total score	Total Tokens input	Total tokens output	Tokens total	Total cost (\$)
anthropic/claude-opus-4.7	9	0	20	29	59 860	2 898	62 758	0.4
x-ai/grok-4.20	3	0	19	22	41 726	1 044	42 770	0.1
openai/gpt-5.4	7	0	16	23	40 850	2 298	43 148	0.2
moonshotai/kimi-k2.5	12	1	19	32	40 223	123 838	164 061	0.4
anthropic/claude-sonnet-4.5	12	0	20	32	47 510	14 818	62 328	0.4
google/gemma-4-31b-it	3	0	20	23	45 116	1 659	46 775	0.1
google/gemini-2.5-pro	11	0	18	29	43 630	101 408	145 038	1.1
qwen/qwen3.5-397b-a17b	10	0	18	28	44 426	158 041	202 467	0.6
deepseek/deepseek-r1	11	0	18	29	40 036	109 607	149 643	0.5
qwen/qwen3.6-plus	13	0	21	34	44 474	96 372	140 846	0.3
openai/gpt-5.4-pro	12	0	18	30	40 890	39 105	79 995	8.3
mistralai/ministral-14b-2512	3	0	15	18	41 961	1 999	43 960	0.1
google/gemini-3.1-pro-preview	10	2	21	33	44 786	69 454	114 240	1
xiaomi/mimo-v2.5-pro	10	0	18	28	48 692	142 398	191 090	0.2
moonshotai/kimi-k2.6	11	0	18	29	40 150	187 910	228 060	0.8
openai/o4-mini-high	10	1	19	30	40 790	63 225	104 015	0.4
stepfun/step-3.5-flash	9	0	15	24	40 097	403 305	443 402	0.2

meta-llama/llama-4-maverick	6	0	19	25	39 115	1 103	40 218	0.1
openai/gpt-oss-120b	10	0	18	28	42 219	22 207	64 426	0.1
nvidia/llama-3.3-nemotron-super-49b-v1.5	9	0	19	28	40 959	122 124	163 083	0.1
microsoft/phi-4	0	0	15	15	42 481	2 800	45 281	0.1

6 Data and analysis

6.1 Overall score

For the final comparison, I used 21 AI models don't counting the 2 excluded models. These 2 models (pixtral-large-2411 and deepseek-v3.2-speciale) were not available for the second phase through OpenRouter, Hugging Face, and Puter, which made it impossible to run the second round with that model. In the end, there were scores for correct numerical, formula and unit answers.

Table 7. Total statistics of all methods

Method	Correct numerical answers	Formula score	Unit score	Total score
Original statements	203 / 525 = 38.7%	18 / 525 = 3.4%	369 / 525 = 70.3%	590 / 1575 = 37.5%
Simplified statements	138 / 525 = 26.3%	12 / 525 = 2.3%	360 / 525 = 68.6%	510 / 1575 = 32.3%
Statements with examples	181 / 525 = 34.5%	6 / 525 = 1.1%	376 / 525 = 71.6%	563 / 1575 = 35.4%

1. The first important and surprising observation is that the original statements produced the best numerical and formula result: 203 and 18 correct numerical and formula answers correspondingly with a solid gap compared to the others.
2. The simplified-statement method gave the weakest result in numerical and unit category, with only 138 and 360 correct answers respectively.
3. The with examples method has the best unit score but it still did not reach the original baseline for numerical category while also having the lowest accuracy in formulas. Though, it performed better than simplified statements way.

Therefore, the first hypothesis — that rewriting problems into a cleaner Given / Find format would improve results — was not supported by the data. Instead of improving

performance, simplified statements reduced numerical accuracy by 65 correct answers compared with the original phase making it the least effective method out of three.

The second hypothesis — that adding solved examples would help language models solve similar problems — was slightly partially supported. Though it performed better than simplified statements method, compared with the original statements, it was 22 numerical answers lower. Although it has the best unit accuracy of 71.6%, it doesn't help it to be the most effective method considering other categories.

It was suggested that new statements would help AI models solve problems, but in this experiment this did not happen. A better way to phrase the explanation is that the original textbook-style wording may have contained useful context or may have been closer to the type of problems the models had seen during training. The simplified statements removed part of this natural context, so even though they were clearer for humans, they may have made it harder for the models to recognize the correct solution pattern. The statements with examples performed better than the simplified statements, but the additional examples may also have increased prompt length and sometimes distracted the models from the target problem.

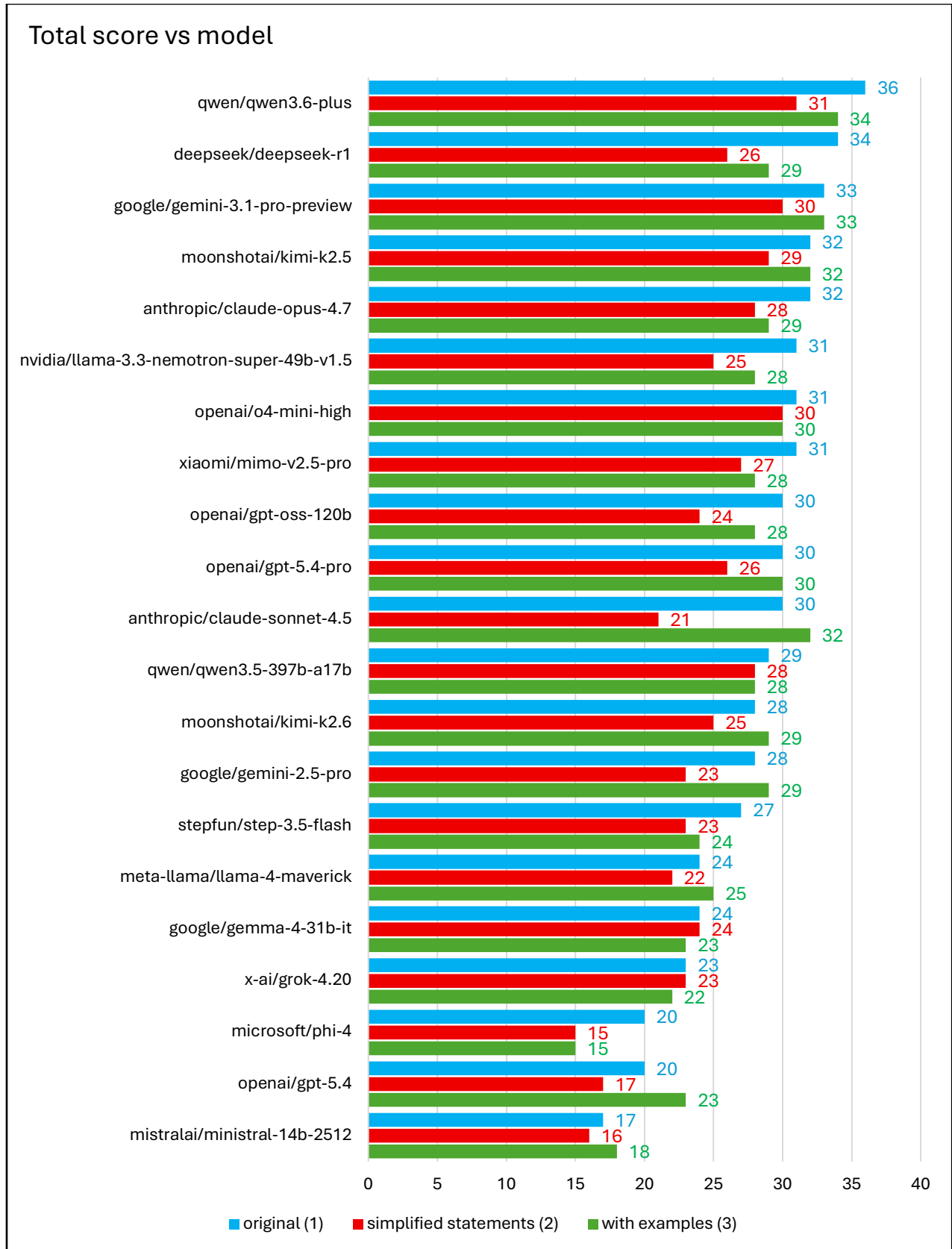


Fig. 2. Total score of all methods vs model

The best overall performing language model by total score was qwen3.6-plus in all 3 methods. This suggests that qwen3.6-plus was the most stable high-performing model across the experiment. However, formula evaluation was not sufficiently reliable because almost all models received zero or one formula point, while unit accuracy was much higher and sometimes inflated the total score. These results should not be interpreted as a final universal model ranking, though it gives a clear surface view about this experiment.

Because of the reasons mentioned above, numerical accuracy was considered the most informative metric for defining the best LLM model for physics problems solving. Chart shown in figure 3 was created to check if the unit score really inflated total score. It presents only total numeric answers for all methods.

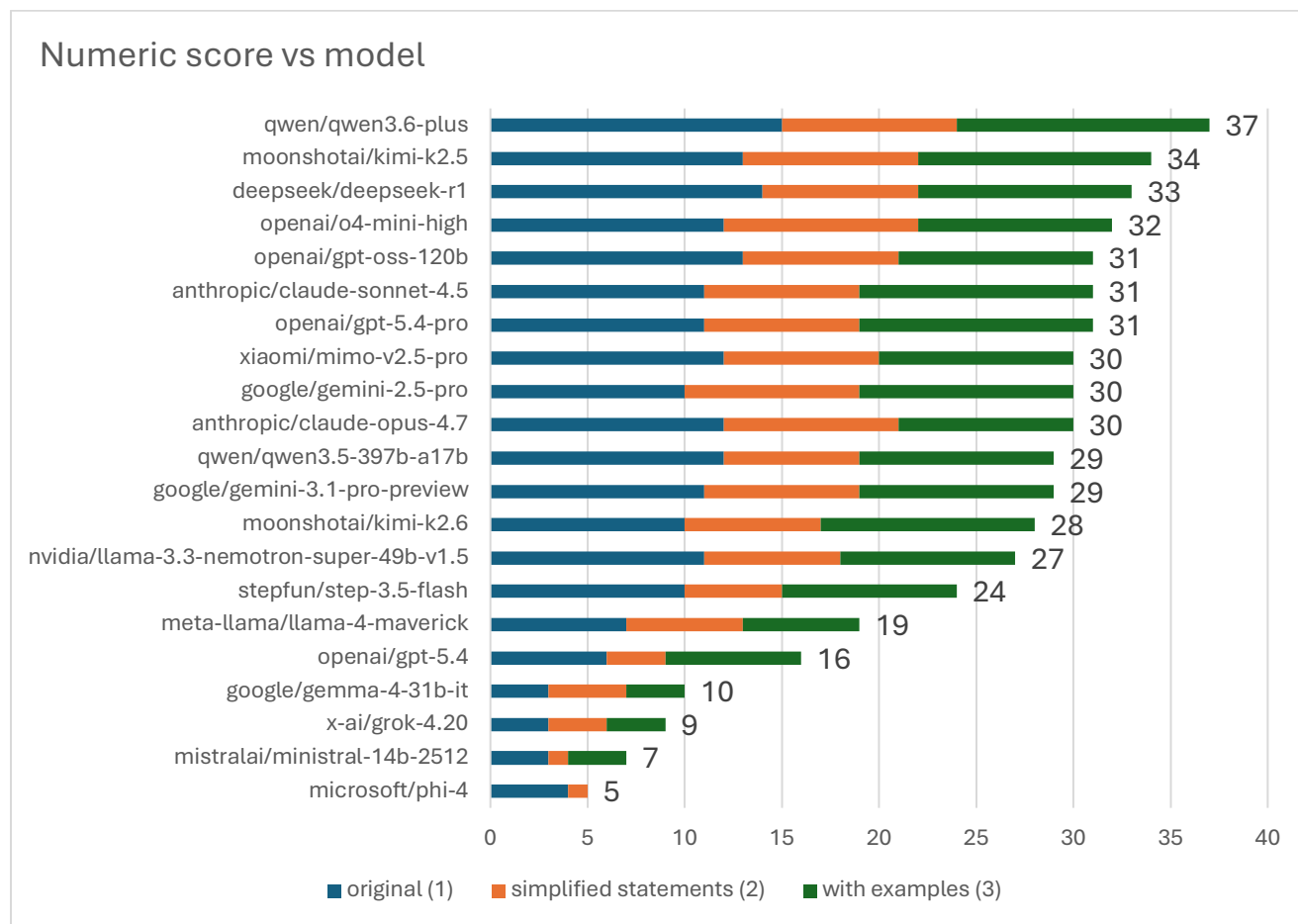


Fig. 3. Total numeric score vs model

The chart reveals that performance was not equally stable across the three methods. For example, DeepSeek R1 decreased from 14 correct answers with the original statements to 6 with simplified statements, before recovering to 9 when examples were added. Qwen 3.6

Plus followed a similar pattern, falling from 15 to 9 and then increasing to 13. In contrast, some language models improved when examples were introduced: Claude Sonnet 4.5 increased from 8 correct answers with simplified statements to 12 with examples, and Kimi K2.6 increased from 7 to 11. This supports the overall result that adding examples partly recovered the accuracy lost during simplification, although for most high-performing models it still did not exceed the original problem format.

The chart also shows that the leading language model did not change. Qwen 3.6 Plus remained the best-performing, achieving the highest total score in Figure 2 and the highest combined numerical score of 37 in Figure 3. This suggests that its first-place position was not caused only by high unit scores; it also performed strongest on the more important numerical-answer metric. However, the ranking of several other models changed when unit and formula points were removed. For example, Kimi K2.5 moved close to the top with 34 correct numerical answers, while some models that had relatively strong total scores ranked lower when only numerical accuracy was considered. Therefore, Figure 3 confirms Qwen 3.6 Plus as the most consistently successful model in this experiment, but it also shows that the total-score metric affected the ranking of the models below first place.

6.2 Language difference

Evaluating models in more than one language is important because model performance may vary across languages and evaluation settings. [5] [6] For the language comparison, I used only the same 21 models that were present. The problem language was classified by source: Russian problems came from the 3,800 Problems for School Students and University Applicants collection, while English problems came from OpenStax-style sources. This gives 14 Russian problems and 11 English problems, or 294 Russian model answers and 231 English model answers per method.

Table 8. numerical accuracy & total score in different languages in 3 different methods

Method	Russian numerical accuracy	English numerical accuracy	Russian total score	English total score
Original statements	92 / 294 = 31.3%	111 / 231 = 48.1%	323 / 882 = 36.6%	267 / 693 = 38.5%
Simplified statements	64 / 294 = 21.8%	74 / 231 = 32.0%	267 / 882 = 30.3%	246 / 693 = 35.5%
With examples	84 / 294 = 28.6%	97 / 231 = 42.0%	309 / 882 = 35.0%	260 / 693 = 37.5%

Total score is relatively similar to each other but in simplified statements method it has a bigger gap of 5.2%. The English problems had higher numerical accuracy in every phase. However, this does not prove that the models are simply “better at English,” because the English and Russian problems were not direct translations of the same tasks. They came from different sources and may have had different difficulty levels. Also, there were less English problems than Russian which made it easier to gain percentage accuracy for English language. Therefore, language, source style, and problem difficulty are mixed.

6.3 Cost

Original problems set:

The total cost was calculated from all requests made to each language model for the original set of 25 problems. The aim of it was to evaluate whether more expensive models provided better problem-solving performance by comparing cost with the number of correct and incorrect numerical answers. The models varied considerably in price, so cost efficiency is an important factor when comparing them. The percentage of accuracy is present next to the total cost on chart:

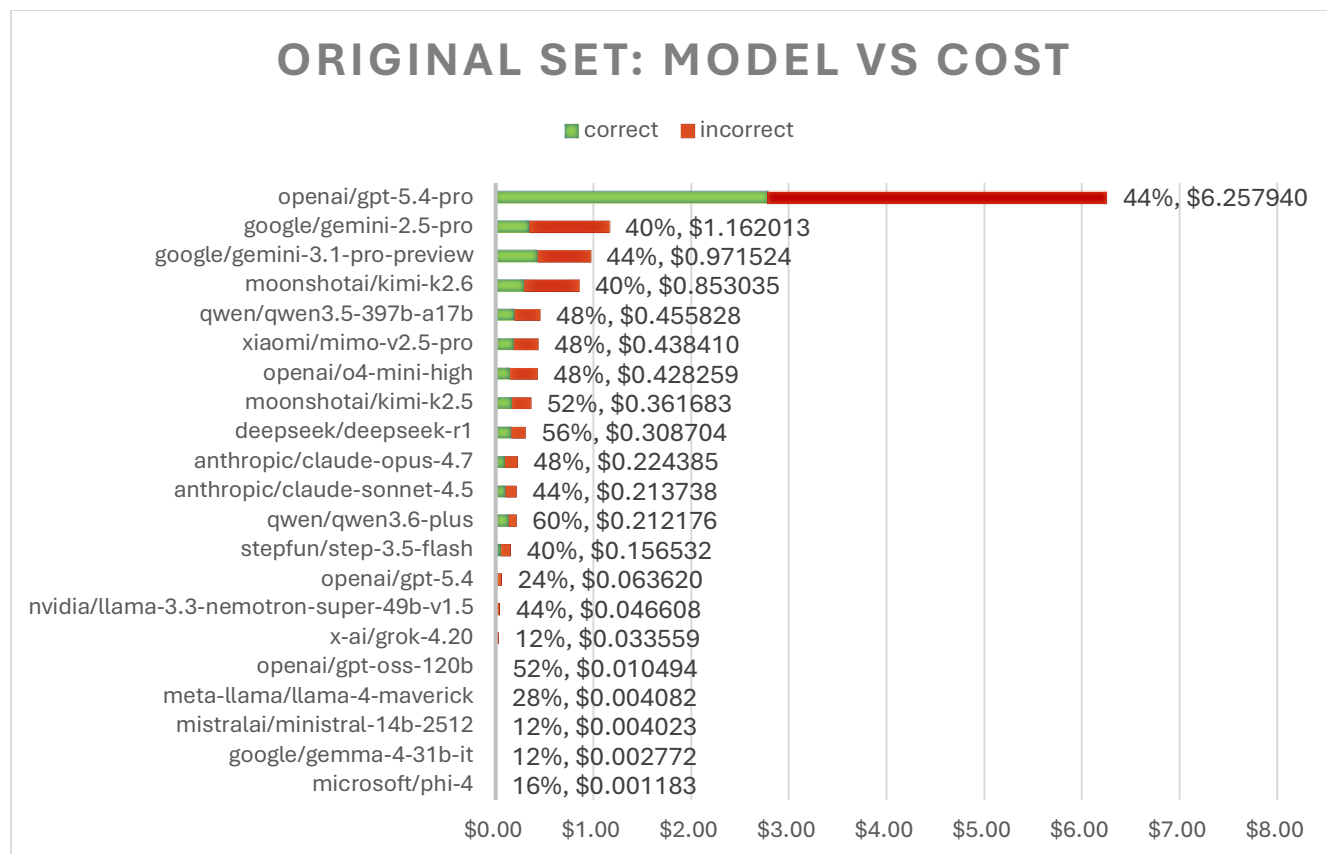


Fig. 4. Model vs cost in original set of problems

The first results show that a higher cost did not consistently correspond to higher numerical accuracy. The most expensive language model, OpenAI GPT-5.4 Pro, cost about \$6.26 for the original problem set, which was considerably more than every other model, but it did not achieve the highest number of correct answers (44%). In comparison, models such as Qwen 3.6 Plus (60%) and DeepSeek R1 (56%) achieved stronger numerical results at a much lower total cost.

There are also several very inexpensive models, with total costs below \$0.10, although their accuracy varies substantially. For example, gpt-5.4 has accuracy of 24% with \$0.06 cost while gpt-oss-120b has 52% accuracy with \$0.01 cost. This suggests that low cost alone does not guarantee good performance either. This chart would be used when compared with other problem sets.

Simplified statements set:

The same cost analysis was carried out for the simplified statements set. The percentage of correctly solved problems is also displayed next to each cost.

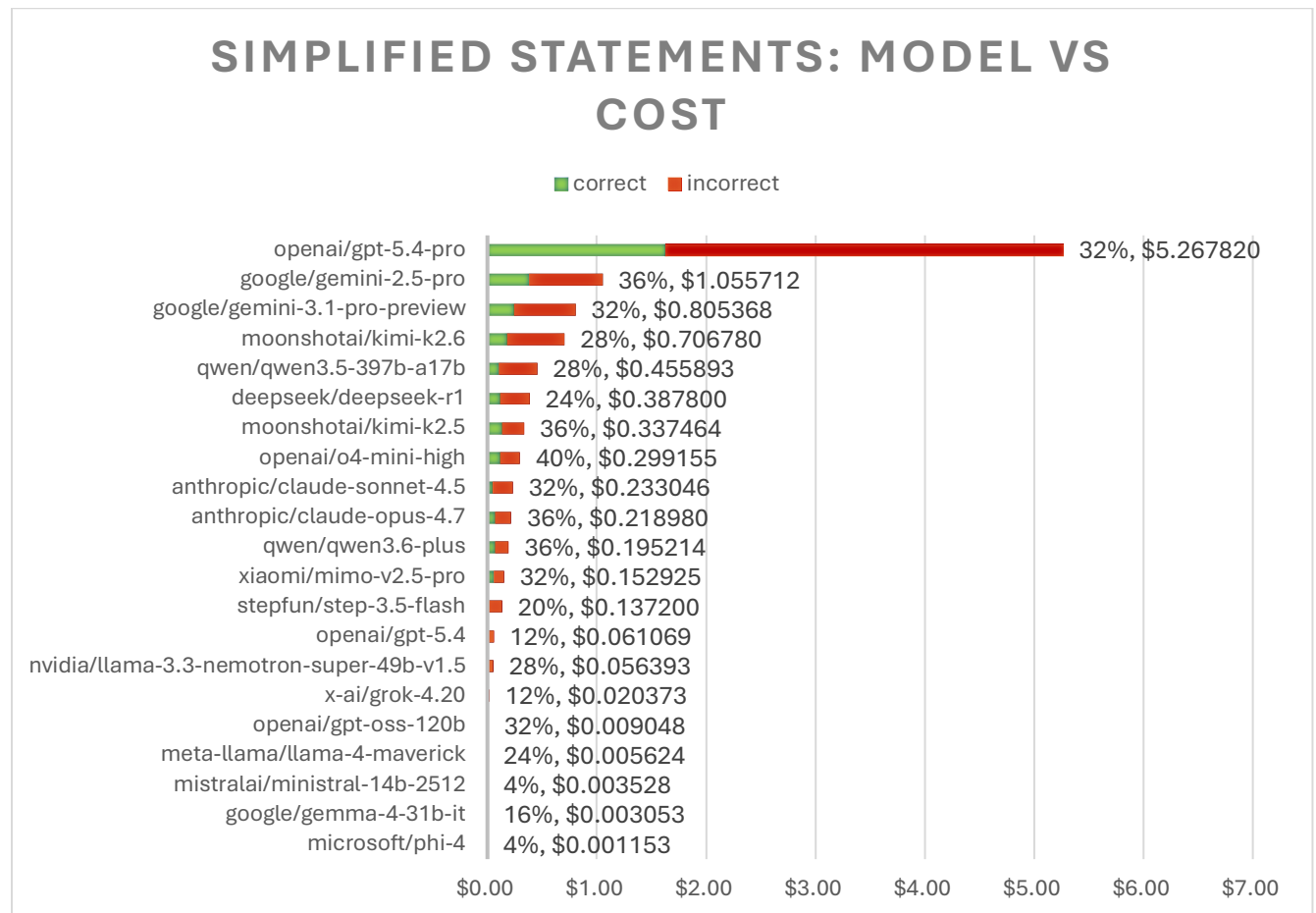


Fig. 5. Model vs cost in simplified statement set of problems

The cost results again show no clear relationship between higher spending and higher accuracy. OpenAI GPT-5.4 Pro remained by far the most expensive language model, costing approximately \$5.27 for the simplified set while achieving 32% numerical accuracy. Several much cheaper models achieved similar or better results. For example, Moonshot Kimi K2.5 reached 36% accuracy at approximately \$0.34, while another model reached 40% accuracy at approximately \$0.30. At the same time, some of the cheapest models achieved relatively low accuracy, showing that lower cost does not automatically mean better cost efficiency.

In comparison with the original set, the simplified phase was cheaper for every model. This can be explained easily: input tokens were reduced. However, this reduction in cost was accompanied by an overall reduction in numerical accuracy. Therefore, the simplified-statement method did not provide a better accuracy cost trade-off.

With examples set:

With examples set of problems contained a lot of input tokens and it was expected that costs were going to be the highest out of all 3 sets. But how much the cost corresponds to the accuracy in the highest costed results?

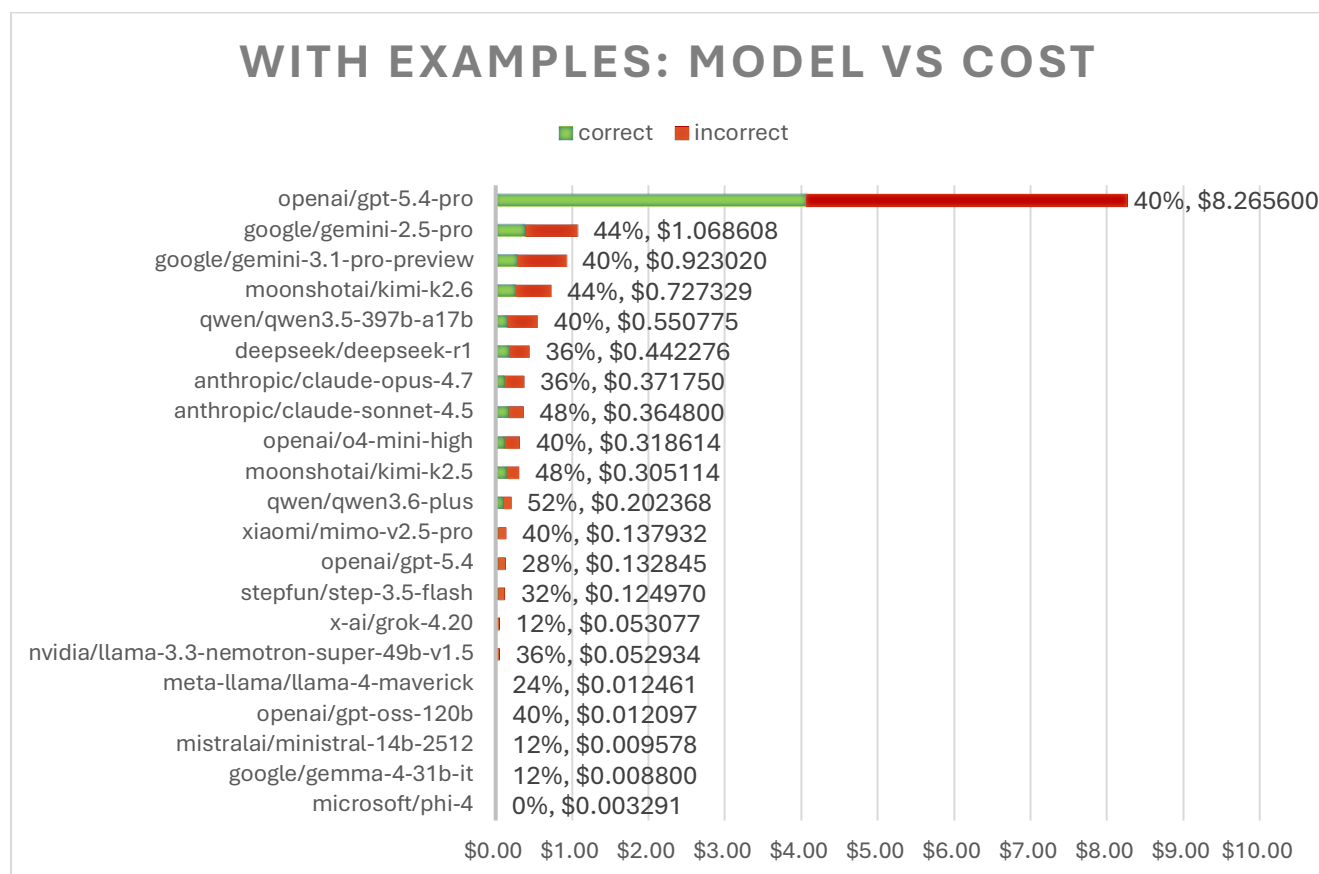


Fig. 6. Model vs cost in with examples

As can be seen, GPT-5.4 Pro was again the most expensive language model, costing approximately \$8.27 with achieving 40% accuracy which is 8% more than in simplified statements set but 4% lower than in the original. This tendency of being better than simplified problems but worse than original can be applied to many other language models

There were also very inexpensive models that produced relatively competitive results. GPT-oss-120b cost approximately \$0.012 and achieved 40% accuracy, while Grok 4.20 also achieved 40% at approximately \$0.053. This indicates that the relationship between price and accuracy was weak: a large increase in expenditure did not reliably produce a corresponding increase in correctly solved problems.

Comparing the three cost charts also shows another consequence of the prompting methods. The simplified statements were generally the cheapest because they contained fewer input tokens, while the examples condition was more expensive because three solved examples were added to every target problem. However, the extra expenditure did not produce the highest accuracy. The examples method improved the overall numerical result compared with simplification, but it remained below the original statements. Therefore, adding more prompt context increased cost substantially without producing a proportional improvement in solving accuracy.

The cost analysis showed that higher spending did not consistently lead to better numerical accuracy. Therefore, selecting a language model should consider both accuracy and cost efficiency rather than only the model's reputation or benchmark performance.

7 Conclusion and evaluation

The results only partially answer the research question. The method with solved examples did improve accuracy compared with the simplified-statement method: overall numerical accuracy increased from 26.3% to 34.5%. This improvement was seen for both Russian and English problems. Therefore, adding relevant solved examples can help AI models recover some accuracy when the task has been rewritten in a more structured format.

However, the experiment did not find a method that improved results beyond the original problem statements. The original phase still had the highest numerical accuracy: 38.7%, compared with 34.5% for the example method and 26.3% for simplified statements. This means the strongest conclusion is that, in this experiment, simplifying physics problems into a Given-Find format did not improve accuracy, and adding examples helped, but not enough to outperform the original wording.

8 Limitations and improvements

1. Data set was small (only 25 problems); thus, every correct answer gave +4% score in a category.
2. Formula rating was too strict, because equivalent formulas can be written in different LaTeX forms.
3. Unit scores were easier to get than numerical answers, so total score may overestimate real solving ability.
4. Some models became unavailable or had provider, timeout, token-limit, or parsing problems.
5. Each model was run once per problem, so random variation in model outputs was not measured. [7] [8]
6. The examples were selected by physics category, but not always closely matched to each target problem.

A stronger investigation should use a larger dataset. It would also be better to test four separate conditions: original statement, original statement with examples, simplified statement, and simplified statement with examples. Points for right defining of unit should be lowered per answer. Repeating each run several times, using fixed model settings, improving formula-equivalence checking, and adding partial human grading would make the results more reliable. [9]

Bibliography

- [1] Yinggan Xu, Yue Liu, Zhiqiang Gao, Changnan Peng, Di Luo, "PhySense: Principle-Based Physics Reasoning Benchmarking for Large Language Models," 30 May 2025. [Online]. Available: <https://arxiv.org/abs/2505.24823>.
- [2] Xiaoxuan Wang, Ziniu Hu, Pan Lu, Yanqiao Zhu, Jieyu Zhang, Satyen Subramaniam, Arjun R. Loomba, Shichang Zhang, Yizhou Sun, Wei Wang, "SchiBench: Evaluating College-Level Scientific Problem-Solving Abilities of Large Language Models," 28 Jun 2024. [Online]. Available: <https://arxiv.org/abs/2307.10635>.
- [3] Jingzhe Ding, Yan Cen, Xinyuan Wei, "Using Large Language Model to Solve and Explain Physics Word Problems Approaching Human Level," 20 Sep 2023. [Online]. Available: <https://arxiv.org/abs/2309.08182>.

- [4] Xinyu Pang, Ruixin Hong, Zhanke Zhou, Fangrui Lv, Xinwei Yang, Zhilong Liang, Bo Han, Changshui Zhang, "Physics Reasoner: Knowledge-Augmented Reasoning for Solving Problems with Large Language Models," 18 Dec 2024. [Online]. Available: <https://arxiv.org/abs/2412.13791>.
- [5] Alena Fenogenova, Artem Chervyakov, Nikita Martynov, Anastasia Kozlova, Maria Tikhonova, Albina Akhmetgareeva, Anton Emelyanov, Denis Shevelev, Pavel Lebedev, Leonid Sinev, Ulyana Isaeva, Katerina Kolomeytseva, Daniil Moskovskiy, Elizaveta Goncharova, Nik, "MERA: A Comprehensive LLM Evaluation in Russian," 2 Aug 2024. [Online]. Available: <https://arxiv.org/abs/2401.04531>.
- [6] Xin Xu, Qiyun Xu, Tong Xiao, Tianhao Chen, Yuchen Yan, Jiaxin Zhang, Shizhe Diao, Can Yang, Yang Wang, "UGPhysics: A Comprehensive Benchmark for Undergraduate Physics Reasoning with Large Language Models," 3 Jun 2025. [Online]. Available: <https://arxiv.org/abs/2502.00334>.
- [7] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, Denny Zhou, "Self-Consistency Improves Chain of Thought Reasoning in Language Models," 7 Mar 2023. [Online]. Available: <https://arxiv.org/abs/2203.11171>.
- [8] Amir Bralin, N. Sanjay Rebello, "AI Reasoning Models for Problem Solving in Physics," 28 Aug 2025. [Online]. Available: <https://arxiv.org/abs/2508.20941>.
- [9] Anka Reuel, Amelia Hardy, Chandler Smith, Max Lamparth, Malcolm Hardy, Mykel J. Kochenderfer, "BetterBench: Assessing AI Benchmarks, Uncovering Issues, and Establishing Best Practices," 20 Nov 2024. [Online]. Available: <https://arxiv.org/abs/2411.12990>.

10 Addendum

9.1 Russian original statements

№	Statement (source)
mechanics	
1	Частица пролетает расстояние $l = 2$ м равномерно, а затем тормозит с ускорением $a = 5 \cdot 10^5$ м/с ² . При какой скорости частицы время движения ее от вылета до остановки будет наименьшим?
2	Точка движется по прямой согласно закону $x = At + Bt^3$, где $A = 6$ м/с; $B = -0,125$ м/с ³ . Определить: среднюю скорость точки в интервале времени от $t_{\{1\}} = 2$ с до $t_{\{2\}} = 6$ с;
4	Закон движения материальной точки имеет вид: $x = a_{\{1\}} + b_{\{1\}} \cdot t + y_{\{1\}} \cdot t^2$; $y = a_{\{2\}} + b_{\{2\}} \cdot t + y_{\{2\}} \cdot t^2$, где $b_{\{1\}} = b_{\{2\}} = 1$ м/с, $y_{\{1\}} = -1$ м/с ² , $y_{\{2\}} = 2$ м/с ² . Каково ее ускорение? Определить угол между скоростью и ускорением в момент времени $t = 1$ с.
gas laws and thermodynamics	
7	Какую работу в кДж совершает при изотермическом расширении водород, взятый при температуре $t = 11$ °С, если его объем увеличивается в 3 раза? Масса водорода $m = 5$ г.
10	Газ, занимающий объем $V_{\{1\}} = 10$ л при давлении $p_{\{1\}} = 2 \cdot 10^5$ Па, расширяется изотермически до объема $V_{\{2\}} = 28$ л. Какую работу в кДж он при этом совершает?
optics	
13	Луч последовательно отражается от двух плоских зеркал, причем луч, падающий на первое зеркало, параллелен плоскости второго зеркала, а дважды отраженный луч параллелен плоскости первого зеркала. Чему равен угол ϕ между зеркалами?
14	На дне сосуда, заполненного водой, лежит плоское зеркало. Человек, наклонившийся над сосудом, видит изображение своего лица в зеркале на расстоянии $d = 25$ см, если расстояние от лица до поверхности воды $h = 5$ см. Определить глубину сосуда l в см.
electrostatics	
16	На концах тонкого непроводящего горизонтального стержня длиной $l = 1$ м закреплены две маленькие бусинки, а третья надета на стержень, по которому она может перемещаться без трения. Всем бусинкам сообщают одинаковые заряды $q = 10^{-6}$ Кл. Найти период малых колебаний подвижной бусинки. Масса бусинки $m = 5$ г.
17	При напряженности электрического поля $E = 10^6$ В/м воздух перестает быть надежным изолятором и в нем происходит искровой разряд. Каким должен быть радиус металлического шара, чтобы на нем мог удержаться заряд $q = 1$ Кл?
18	Электрон движется в направлении силовой линии однородного электрического поля, напряженность которого $E = 100$ В/м. Какое расстояние пролетит он до полной остановки в см, если начальная скорость электрона $v = 10^6$ м/с?
19	Два одинаковых воздушных конденсатора соединены последовательно и подключены к батарее с постоянной ЭДС. Один из них заполняют диэлектриком с диэлектрической проницаемостью $\epsilon = 4$. Во сколько раз изменится напряженность электрического поля в этом конденсаторе?
nuclear	
21	При переходе электронов с некоторой более удаленной орбиты на вторую боровскую орбиту, атомы водорода испускают монохроматический пучок света. При падении этого пучка по нормали на дифракционную решетку максимум второго порядка ($k = 2$) наблюдается под углом дифракции $\phi = 30^\circ$. Постоянная дифракционной решетки $d = 2,6$ мкм. Определить номер орбиты n , на котором первоначально находились электроны в атомах водорода.

22	Подводная лодка «Наутилус» (США) имеет мощность топливных установок $P = 14,7$ МВт, КПД $\eta = 25\%$. Топливом служит обогащенный уран массой $m_{\{0\}} = 1$ кг, при делении ядер которого выделяется энергия $E = 6,9 \cdot 10^{13}$ Дж. Определить запас горючего, необходимого для годового плавания лодки.
23	При определении периода полураспада короткоживущего радиоактивного изотопа использовался счетчик импульсов. За минуту в начале наблюдения было насчитано $\Delta(n_{\{0\}}) = 250$ импульсов, а через время $t = 1$ ч было зарегистрировано $\Delta(n) = 92$ импульса. Чему равен период полураспада в мин данного изотопа?

9.2 First script:

```
import os
import re
import json
import time
import csv
import requests
import sys
from pathlib import Path
from google.colab import files
from datetime import datetime

class Logger(object):
    def __init__(self, filename):
        self.terminal = sys.stdout
        self.log = open(filename, "a", encoding="utf-8")

    def write(self, message):
        # This writes to the console
        self.terminal.write(message)
        # This writes to the file
        self.log.write(message)
        self.log.flush()

    def flush(self):
        self.terminal.flush()

    def close(self):
        self.log.close()

def read_env_value(env_path, key_name):
    env_path = Path(env_path)
    if not env_path.exists():
        return None

    for raw_line in env_path.read_text(encoding="utf-8").splitlines():
        line = raw_line.strip()
        if not line or line.startswith("#"):
            continue
        if line.startswith("export "):
            line = line[len("export "):].strip()
        if "=" not in line:
            continue

        key, value = line.split("=", 1)
        if key.strip() != key_name:
            continue

        value = value.strip()
        if len(value) >= 2 and value[0] == value[-1] and value[0] in ('"', "'"):
            value = value[1:-1]
        return value

    return None
```

```

PROBLEM_MAX = 25
PROBLEM_START = 1
PROMPT_VERSION = 0 # first version, taking from PROMPT_TEMPLATES[] (below)

INPUT_CSV = "/content/Physics Problems with examples.csv"

OUTPUT_FOLDER = "/content/"

OUTPUT_FILENAME = "physics_experiment"

AI_MODELS = [
    "anthropic/claude-opus-4.7"
    #"anthropic/claude-sonnet-4.5"
    #"deepseek/deepseek-r1"
    #"deepseek/deepseek-v3.2-speciale"
    #"google/gemini-2.5-pro"
    #"google/gemini-3.1-pro-preview"
    #"google/gemma-4-31b-it"
    #"meta-llama/llama-4-maverick"
    #"microsoft/phi-4"
    #"mistralai/ministral-14b-2512"
    #"mistralai/pixtral-large-2411"
    #"moonshotai/kimi-k2.5"
    #"moonshotai/kimi-k2.6"
    #"nvidia/llama-3.3-nemotron-super-49b-v1.5"
    #"openai/gpt-5.4-pro"
    #"openai/gpt-5.4"
    #"openai/gpt-oss-120b"
    #"openai/o4-mini-high"
    #"qwen/qwen3.5-397b-a17b"
    #"qwen/qwen3.6-plus"
    #"stepfun/step-3.5-flash"
    #"x-ai/grok-4.20"
    #"xiaomi/mimo-v2.5-pro"
]

#####

OPENROUTER_API_KEY = ""

AI_MODEL_DEFAULT_TEMPERATURE = 0.1
API_URL = "https://openrouter.ai/api/v1/chat/completions"

REQUEST_TIMEOUT = 1800
MAX_RETRIES = 2
SLEEP_BETWEEN_REQUESTS_SEC = 1.0

REQUIRED_OUTPUT_FIELDS = ["final_numeric", "final_formula", "final_units"]

if not OPENROUTER_API_KEY:
    raise ValueError(
        f"Could not load OpenRouter API key. Expected OPENROUTER_API_KEY"
    )

PROMPT_TEMPLATES = [
    # version 1 (index 0)
    """
You are an expert in physics problems.

# INPUT

The following is a physics problem that needs to be solved.

{{PROBLEM}}

# OUTPUT REQUIREMENTS

```

Answer must be only in JSON format. Do not use markdown. Do not add any extra text.

Return a single JSON object with these required keys:

- "final_numeric": the final numeric value as a JSON number (not a string). If the answer is symbolic-only, use null.

- "final_formula": a string containing the final answer as a formula/expression written as valid LaTeX.

- "final_units": a string with units in International System of Units (SI) that was used for the numeric answer.

If answer is unitless, use "1".

If the numeric answer is null, still provide the expected units for the expression.

If the problem does not explicitly specify units, default to SI units.

IMPORTANT: answer only with JSON, do not add any comments or other text except output JSON with results.

OUTPUT FORMAT - EXAMPLE 1:

```
{
  "final_numeric": 1,
  "final_formula": "LATEX_FORMULA_INSERT_HERE",
  "final_units": "N" // NEWTON units
}
```

OUTPUT FORMAT - EXAMPLE 2:

```
{
  "final_numeric": 1,
  "final_formula": "LATEX_FORMULA_INSERT_HERE",
  "final_units": "J" // JOULE units
}
```

OUTPUT FORMAT - EXAMPLE 3:

```
{
  "final_numeric": 1,
  "final_formula": "LATEX_FORMULA_INSERT_HERE",
  "final_units": "kg*m/s^2" // kilogram multiplied by meters per second squared
}
```

INCORRECT OUTPUT FORMAT EXAMPLE 1:

```
{
  "final_numeric": "zero",
  "final_formula": "m / divide by n",
  "final_units": "KILOGRAMS"
}
```

INCORRECT OUTPUT FORMAT EXAMPLE 1:

```
{
  "final_numeric": [11,0],
  "final_formula": "m / divide by n",
  "final_units": "KILOGRAMS"
}
```

```
"""
# version 2 goes here
"""
"""
]
```

```
def normalize_header(value):
    return str(value).strip().lower() if value is not None else ""
```

```
def load_problems_from_file(csv_path, start_problem=1, max_items=25):
    items = []
```

```
    print(f>Loading problems from {csv_path}, start_problem={start_problem}, max_items={max_items}")
```

```

with open(csv_path, "r", encoding="utf-8-sig", newline="") as f:
    reader = csv.DictReader(f)

    if reader.fieldnames is None:
        raise ValueError("CSV file is empty or missing a header row.")

    statement_field = None
    for field in reader.fieldnames:
        if normalize_header(field) == "statement":
            statement_field = field
            break

    if statement_field is None:
        raise KeyError(f'Column "Statement" not found. Headers seen: {reader.fieldnames}')

    # i is the index of row in CSV ()
    for i, row in enumerate(reader):
        # check if have statement is not empty
        statement = row.get(statement_field)

        if statement is None:
            continue

        # also skip if not reached start problem yet
        if i < start_problem-1:
            continue

        statement = str(statement).strip()
        if not statement:
            continue

        items.append({
            # we add +1 because have first row is HEADER (1 row)
            "statement_row_id": i+1,
            "statement": statement
        })

        if len(items) >= max_items:
            break

    if not items:
        raise ValueError('No non-empty rows found from the specified start item in the "Statement" column.')

    return items

def build_user_prompt(problem_text):
    return PROMPT_TEMPLATES[PROMPT_VERSION].replace("{{PROBLEM}}", problem_text)

def extract_message_text(content):
    if isinstance(content, str):
        return content.strip()
    if isinstance(content, list):
        parts = []
        for item in content:
            if isinstance(item, str):
                parts.append(item)
            elif isinstance(item, dict):
                if item.get("type") == "text":
                    parts.append(item.get("text", ""))
                elif "text" in item:
                    parts.append(str(item["text"]))
        return "\n".join(p for p in parts if p).strip()
    return str(content).strip()

PARSING_ERROR_CODE = "PARSING_ERROR"

def _append_parse_error(obj, code=PARSING_ERROR_CODE):
    if not isinstance(obj, dict):

```

```

        return obj

existing = str(obj.get("error", "") or "").strip()
parts = [p.strip() for p in existing.splitlines() if p.strip()]

if code not in parts:
    parts.append(code)

obj["error"] = "\n".join(parts)
return obj

def _strip_json_code_fence(text):
    text = str(text).strip()

    fenced = re.match(
        r"```(?:json)?\s*(.*?)\s*```$",
        text,
        flags=re.IGNORECASE | re.DOTALL
    )
    if fenced:
        return fenced.group(1).strip()

    text = re.sub(r"```(?:json)?\s*", "", text, flags=re.IGNORECASE)
    text = re.sub(r"\s*```$", "", text)
    return text.strip()

def _escape_invalid_json_backslashes(text):
    r"""
    Repair common LLM JSON mistakes in LaTeX strings.

    Valid JSON escapes are preserved:
    \", \\, \/, \b, \f, \n, \r, \t, \u1234

    Invalid LaTeX-style escapes are repaired:
    \frac -> \\frac
    \text -> \\text
    \rho -> \\rho
    \_ -> \\_
    """
    out = []
    i = 0
    hex_chars = set("0123456789abcdefABCDEF")

    while i < len(text):
        ch = text[i]

        if ch != "\\":
            out.append(ch)
            i += 1
            continue

        if i + 1 >= len(text):
            out.append("\\\\")
            i += 1
            continue

        nxt = text[i + 1]

        if nxt in ['"', '\\', '/', "b", "f", "n", "r", "t"]:
            out.append("\\\" + nxt)
            i += 2
        elif (
            nxt == "u"
            and i + 5 < len(text)
            and all(c in hex_chars for c in text[i + 2:i + 6])
        ):
            out.append(text[i:i + 6])
            i += 6
        else:
            out.append("\\\\")
            i += 1

```

```

    return "".join(out)

def _raw_decode_json_prefix(text):
    r"""
    Parse a JSON object from the beginning of the response.

    Unlike json.loads(), JSONDecoder.raw_decode() can parse this case:

        {"final_numeric": 1, ...}
        Wait, let me recalculate...

    The object is marked with PARSING_ERROR because the full response was
    not valid JSON-only output.
    """
    decoder = json.JSONDecoder()

    candidates = [text]
    repaired = _escape_invalid_json_backslashes(text)
    if repaired != text:
        candidates.append(repaired)

    for candidate in candidates:
        try:
            obj, end = decoder.raw_decode(candidate)
            if isinstance(obj, dict):
                if candidate[end:].strip():
                    _append_parse_error(obj)
                if candidate != text:
                    _append_parse_error(obj)
            return obj
        except json.JSONDecodeError:
            pass

    return None

# we use this function cleanup JSON input string like this:
#{
# "final_numeric": 0.32,
# "final_formula": "\\frac{d \\cdot v_{truck}}{v_{truck}^2 + v_{car}^2}",
# "final_units": "h"
#}
# it failed to be parsed as JSON because of LaTeX formular having \\frac instead of \\frac

def _extract_first_balanced_json_object(text):
    r"""
    Return the first balanced {...} block without using a greedy regex.

    This intentionally ignores braces inside JSON strings, so LaTeX like
    "\\frac{a}{b}" does not break object-boundary detection.
    """
    for start in [m.start() for m in re.finditer(r"\\{", text)]:
        depth = 0
        in_string = False
        escaped = False

        for i in range(start, len(text)):
            ch = text[i]

            if in_string:
                if escaped:
                    escaped = False
                elif ch == "\\":
                    escaped = True
                elif ch == "'":
                    in_string = False
            else:
                if ch == "'":
                    in_string = True
                elif ch == "{":
                    depth += 1
                elif ch == "}":

```

```

        depth -= 1
        if depth == 0:
            return text[start:i + 1]

    return None

def _loads_json_object_candidate(candidate):
    for fixed in (candidate, _escape_invalid_json_backslashes(candidate)):
        try:
            obj = json.loads(fixed)
            if isinstance(obj, dict):
                return obj
        except json.JSONDecodeError:
            pass

    return None

def _extract_quoted_value(text, start_quote_index):
    chars = []
    escaped = False
    i = start_quote_index + 1

    while i < len(text):
        ch = text[i]

        if escaped:
            chars.append(ch)
            escaped = False
        elif ch == "\\":
            chars.append(ch)
            escaped = True
        elif ch == '"':
            return "".join(chars), i + 1
        else:
            chars.append(ch)

        i += 1

    # Response was truncated before the closing quote.
    return "".join(chars), len(text)

def _extract_bracketed_value(text, start_index):
    opening = text[start_index]
    closing = "]" if opening == "[" else "}"

    depth = 0
    in_string = False
    escaped = False

    for i in range(start_index, len(text)):
        ch = text[i]

        if in_string:
            if escaped:
                escaped = False
            elif ch == "\\":
                escaped = True
            elif ch == '"':
                in_string = False
        else:
            if ch == '"':
                in_string = True
            elif ch == opening:
                depth += 1
            elif ch == closing:
                depth -= 1
                if depth == 0:
                    raw = text[start_index:i + 1]
                    try:
                        return json.loads(_escape_invalid_json_backslashes(raw)), i + 1
                    except Exception:
                        return raw, i + 1

```

```

# Response was truncated before the bracket closed.
return text[start_index:], len(text)

def _extract_value_after_key(text, key):
    match = re.search(r'"'+ re.escape(key) + r'"\\s*:', text)
    if not match:
        return None, False

    i = match.end()

    while i < len(text) and text[i].isspace():
        i += 1

    if i >= len(text):
        return None, False

    if text[i] == '"':
        value, _ = _extract_quoted_value(text, i)
        return value, True

    if text[i] in "[{":
        value, _ = _extract_bracketed_value(text, i)
        return value, True

    rest = text[i:]
    scalar_match = re.match(
        r"(null|true|false|-?\d+(?:\.\d+)?(?:[eE][+-]?\d+)?)",
        rest
    )
    if scalar_match:
        raw = scalar_match.group(1)
        try:
            return json.loads(raw), True
        except Exception:
            return raw, True

    raw = re.split(r",|\n|\\}", rest, maxsplit=1)[0].strip()
    return raw, bool(raw)

def _salvage_required_fields(text):
    """
    Last-resort partial extraction for malformed/truncated objects.

    This is intentionally conservative: it only extracts the required output
    keys and then lets check_fields() validate missing/wrongly typed fields.
    """
    obj = {}

    for key in REQUIRED_OUTPUT_FIELDS:
        value, found = _extract_value_after_key(text, key)
        if found:
            obj[key] = value

    if obj:
        return _append_parse_error(obj)

    return None

def find_and_extract_json_object_from_text(text):
    text = _strip_json_code_fence(text)

    # 1) Strict parse. Clean JSON-only responses stay clean.
    try:
        obj = json.loads(text)
        if isinstance(obj, dict):
            return obj
    except json.JSONDecodeError:
        pass

```

```

# 2) Parse JSON at the start, even if the model added explanation after it.
obj = _raw_decode_json_prefix(text)
if obj is not None:
    return obj

# 3) Extract the first balanced JSON object without greedy "{.*}" matching.
candidate = _extract_first_balanced_json_object(text)
if candidate:
    obj = _loads_json_object_candidate(candidate)
    if obj is not None:
        return _append_parse_error(obj)

# 4) Last resort: salvage required fields from malformed/truncated JSON.
return _salvage_required_fields(text)

def check_fields(obj):
    if obj is None:
        print("Obj is None?")
        return False

    if "error" not in obj:
        obj["error"] = ""

    error_message = obj["error"]

    for key in REQUIRED_OUTPUT_FIELDS:
        if key not in obj:
            error_message = error_message + "\n" + "MISSING_OUTPUT_FIELD:" + key
            print(f"MISSING_OUTPUT_FIELD Missing required key: {key}")

    num = obj.get("final_numeric")
    if isinstance(num, str):
        stripped = num.strip()
        if stripped.lower() == "null":
            obj["final_numeric"] = None
        else:
            try:
                n = float(stripped)
                if n.is_integer(): n = int(n)
                obj["final_numeric"] = n
            except:
                error_message += "\nFINAL_NUMERIC_IS_NOT_NUMBER"
    elif num is not None and not isinstance(num, (int, float)):
        error_message += "\nFINAL_NUMERIC_IS_NOT_NUMBER"
    obj["final_numeric"] = str(num)

    if not isinstance(obj.get("final_formula"), str):
        error_message += "\nFINAL_FORMULA_IS_NOT_STRING"
        obj["final_formula"] = str(obj.get("final_formula", ""))

    if not isinstance(obj.get("final_units"), str):
        error_message += "\nFINAL_UNITS_IS_NOT_STRING"
        obj["final_units"] = str(obj.get("final_units", ""))

    obj["error"] = error_message.strip()
    return True

def call_ai(ai_model, problem_id, problem_text, api_key):
    if not api_key or api_key == "PASTE YOUR OPENROUTER_API_KEY_HERE":
        raise ValueError("Please set OPENROUTER_API_KEY first.")

    payload = {
        "model": ai_model,
        "temperature": AI_MODEL_DEFAULT_TEMPERATURE,
        "messages": [{"role": "user", "content": build_user_prompt(problem_text)}]
    }
    headers = {"Authorization": f"Bearer {api_key}", "Content-Type": "application/json"}
    last_error = None

```

```

for attempt in range(1, MAX_RETRIES + 1):
    try:

        print(f"sending attempt: {attempt} of {MAX_RETRIES+1}")

        # sending request to AI
        response = requests.post(API_URL, headers=headers, json=payload, timeout=REQUEST_TIMEOUT)
        # response.raise_for_status()
        # got data back
        # data = response.json()

        response = requests.post(API_URL, headers=headers, json=payload, timeout=REQUEST_TIMEOUT)

        if response.status_code >= 400:
            print("STATUS:", response.status_code)
            print("BODY:", response.text)
            print("PAYLOAD:", json.dumps(payload, ensure_ascii=False, indent=2))
            response.raise_for_status()

        data = response.json()

        raw_text = extract_message_text(data["choices"][0]["message"]["content"])
        print(f"problem id: {problem_id}, raw response extracted.")

        parsed = find_and_extract_json_object_from_text(raw_text)

        if parsed is None:
            print(f"JSON can not be parsed: {raw_text}")
            return {"ok": False, "parsed": None, "raw_response_text": raw_text, "error": "JSON_PARSE_ERROR"}

        is_valid = check_fields(parsed)

        return {
            "ok": is_valid,
            "parsed": parsed,
            "raw_response_text": raw_text,
            "full_api_response": data,
            "prompt_tokens": data["usage"]["prompt_tokens"],
            "completion_tokens": data["usage"]["completion_tokens"],
            "total_tokens": data["usage"]["total_tokens"],
            "prompt_version": PROMPT_VERSION
        }

    except Exception as e:
        last_error = e
        time.sleep(min(2 ** attempt, 20))

return {"ok": False, "parsed": None, "raw_response_text": None, "error": str(last_error)}

def save_results_csv_and_jsonl(results, csv_path, jsonl_path):

    # saving to JSONL
    with open(jsonl_path, "w", encoding="utf-8") as f:
        for row in results:
            f.write(json.dumps(row, ensure_ascii=False) + "\n")

    fieldnames = [
        "datetime",
        "ai_model",
        "statement_row_id",
        "statement",
        "ok",
        "final_numeric",

```

```

        "final_formula",
        "final_units",
        "error",
        "raw_response_text",
        "full_api_response",
        "prompt_tokens",
        "completion_tokens",
        "total_tokens",
        "prompt_version"
    ]

# saving to CSV
with open(csv_path, "w", encoding="utf-8", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=fieldnames)
    writer.writeheader()
    for row in results:
        writer.writerow({
            "datetime": row.get("datetime"),
            "ai_model": row.get("ai_model"),
            "statement_row_id": row.get("statement_row_id"),
            "statement": row.get("statement"),
            "ok": row.get("ok"),
            "final_numeric": row.get("final_numeric"),
            "final_formula": row.get("final_formula"),
            "final_units": row.get("final_units"),
            "error": row.get("error"),
            "raw_response_text": row.get("raw_response_text"),
            "full_api_response": row.get("full_api_response"),
            "prompt_tokens": row.get("prompt_tokens"),
            "completion_tokens": row.get("completion_tokens"),
            "total_tokens": row.get("total_tokens"),
            "prompt_version": row.get("prompt_version")

        })

# Setup Logging
now = datetime.now()
datetime_string = now.strftime("%Y-%m-%d-%H-%M-%S")

# subfolder name
current_subfolder = Path(OUTPUT_FOLDER) / datetime_string

# Create the directory structure
# parents=True: creates all intermediate folders (e.g., 'parent_folder' and 'subfolder')
# exist_ok=True: prevents an error if the folders already exist
current_subfolder.mkdir(parents=True, exist_ok=True)

log_file = current_subfolder / f"{datetime_string}_{OUTPUT_FILENAME}.log"
file_csv = current_subfolder / f"{datetime_string}_{OUTPUT_FILENAME}.csv"
file_jsonl = current_subfolder / f"{datetime_string}_{OUTPUT_FILENAME}.jsonl"

sys.stdout = Logger(log_file)

try:
    problems = load_problems_from_file(INPUT_CSV, start_problem=PROBLEM_START, max_items=PROBLEM_MAX)
    print(f"Loaded {len(problems)} problems from the Statement column.")
    results = []

    for idx, item in enumerate(problems, start=1):
        statement_row_id = item["statement_row_id"]
        statement = item["statement"]

        print(f"[{idx}/{len(problems)}] Sending problem #{statement_row_id} (csv row {statement_row_id+1})..")

        for ai_model in AI_MODELS:
            api_result = call_ai(
                ai_model, # ai model
                statement_row_id, # problem id from CSV file (first column)
                statement, # problem statement
                OPENROUTER_API_KEY
            )

```

```

now = datetime.now()
formatted_datetime = now.strftime("%Y-%m-%d-%H-%M-%S")

if api_result["ok"]:
    parsed = api_result["parsed"]
    out = {
        "ok": True,
        "datetime": formatted_datetime,
        "ai_model": ai_model,
        "statement_row_id": statement_row_id,
        "statement": statement,
        "final_numeric": parsed.get("final_numeric"),
        "final_formula": parsed.get("final_formula"),
        "final_units": parsed.get("final_units"),
        "error": parsed["error"],
        "raw_response_text": api_result.get("raw_response_text"),
        "full_api_response": api_result.get("full_api_response"),
        "prompt_tokens": api_result.get("prompt_tokens"),
        "completion_tokens": api_result.get("completion_tokens"),
        "total_tokens": api_result.get("total_tokens"),
        "prompt_version": PROMPT_VERSION
    }
    print(" OK:", out)
else:
    out = {
        "ok": False,
        "datetime": formatted_datetime,
        "ai_model": ai_model,
        "statement_row_id": statement_row_id,
        "statement": statement,
        "final_numeric": None,
        "final_formula": None,
        "final_units": None,
        "error": api_result.get("error"),
        "raw_response_text": api_result.get("raw_response_text"),
        "full_api_response": api_result.get("full_api_response"),
        "prompt_tokens": api_result.get("prompt_tokens"),
        "completion_tokens": api_result.get("completion_tokens"),
        "total_tokens": api_result.get("total_tokens"),
        "prompt_version": PROMPT_VERSION
    }
    print(" FAILED:", out["error"])

results.append(out)
time.sleep(SLEEP_BETWEEN_REQUESTS_SEC)

# save csv and jsonl to files
save_results_csv_and_jsonl(results, file_csv, file_jsonl)

finally:
    # Always restore original stdout and download log
    logger_inst = sys.stdout
    sys.stdout = logger_inst.terminal
    logger_inst.close()
    files.download(log_file)
    print("LOG file saved to: ", log_file)

    if 'file_csv' in locals():
        files.download(file_csv)
        files.download(file_jsonl)
        print("CSV saved to:", file_csv)
        print("JSONL saved to:", file_jsonl)

print("\nDone.")

```

9.3 Second script

```

import pandas as pd
import numpy as np

```

```

import re
import json
import ast
from pathlib import Path

try:
    from google.colab import files
except ImportError:
    files = None

RESULTS_DIR = Path("results")
COMBINED_CSV = "combined_csv_files.csv"
ANSWERS_CSV = "Physics Problems 2026 march 7 - Sheet1.csv"
OUTPUT_CSV = "physics_results_with_usage.csv"

if not RESULTS_DIR.exists():
    raise FileNotFoundError(
        f"Could not find the results folder: {RESULTS_DIR.resolve()}\n"
        "Create a folder named 'results' and put the individual result CSV"
        "files inside it."
    )

csv_paths = sorted(
    p for p in RESULTS_DIR.glob("*.csv")
    if p.name not in {COMBINED_CSV, ANSWERS_CSV, OUTPUT_CSV}
)

if not csv_paths:
    raise FileNotFoundError(
        f"No CSV files found in {RESULTS_DIR.resolve()}\n"
        "Put the individual result CSV files inside the 'results' folder."
    )

frames = []
for csv_path in csv_paths:
    df = pd.read_csv(csv_path)

    # Keep the model/source traceable after concatenation.
    # If the CSV has no ai_model/model/source_file column, source_file will
    be used later.
    lower_cols = {str(c).lower().strip() for c in df.columns}
    if not {"ai_model", "model", "source_file"}.intersection(lower_cols):
        df["source_file"] = csv_path.stem

    frames.append(df)

results = pd.concat(frames, ignore_index=True, sort=False)
results.to_csv(COMBINED_CSV, index=False)

if not Path(ANSWERS_CSV).exists():

```

```

    if files is None:
        raise FileNotFoundError(
            f"Could not find {ANSWERS_CSV}. Put it next to this
notebook/script and rerun."
        )
    print(f"Upload the answer-key CSV now: {ANSWERS_CSV}")
    files.upload()

if not Path(ANSWERS_CSV).exists():
    raise FileNotFoundError(
        f"Still could not find {ANSWERS_CSV}. "
        "Check that the uploaded answer-key filename matches exactly."
    )

answers = pd.read_csv(ANSWERS_CSV)

print(f"Merged {len(csv_paths)} CSV files from {RESULTS_DIR}/ into
{COMBINED_CSV}")
print("Merged files:")
for csv_path in csv_paths:
    print(f"- {csv_path}")

print("\nCombined CSV columns:")
print(list(results.columns))
print("\nPhysics answer CSV columns:")
print(list(answers.columns))

print("\nRows in combined CSV:", len(results))
print("Rows in answer CSV:", len(answers))

def normalize_colnames(df):
    df = df.copy()
    df.columns = [str(c).strip() for c in df.columns]
    return df

def find_col(df, candidates, label):
    """Find a column using exact case-insensitive aliases."""
    lower_map = {str(c).lower().strip(): c for c in df.columns}
    for cand in candidates:
        key = str(cand).lower().strip()
        if key in lower_map:
            return lower_map[key]
    raise KeyError(
        f"Could not find column for {label}. Tried: {candidates}. "
        f"Available columns: {list(df.columns)}"
    )

def find_col_optional(df, candidates):
    lower_map = {str(c).lower().strip(): c for c in df.columns}
    for cand in candidates:

```

```

        key = str(cand).lower().strip()
        if key in lower_map:
            return lower_map[key]
    return None

def numeric_to_2dp(value):
    """Extract the first numeric value and round to 2 decimal places."""
    if pd.isna(value):
        return np.nan
    text = str(value).strip()
    if text == '' or text.lower() in {'nan', 'null', 'none'}:
        return np.nan
    match = re.search(r'[-+]?[d*\.]?[d+](?:[eE] [-+]?[d+])?', text)
    if not match:
        return np.nan
    try:
        return round(float(match.group(0)), 2)
    except ValueError:
        return np.nan

def clean_formula(value):
    """Remove leading assignments like x = ..., v_final = ..., answer = ...
    and normalize spaces."""
    if pd.isna(value):
        return ''
    text = str(value).strip()
    if text.lower() in {'nan', 'null', 'none'}:
        return ''
    text = re.sub(r'^\s*[A-Za-z][A-Za-z0-9_]*\s*=\s*', '', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text

def parse_full_api_response(value):
    if isinstance(value, dict):
        return value
    if pd.isna(value):
        return {}
    text = str(value).strip()
    if text == '' or text.lower() in {'nan', 'null', 'none'}:
        return {}
    for parser in (ast.literal_eval, json.loads):
        try:
            parsed = parser(text)
            if isinstance(parsed, dict):
                return parsed
            if isinstance(parsed, str):
                parsed_again = parser(parsed)
                if isinstance(parsed_again, dict):
                    return parsed_again
        except Exception:
            pass
    return {}

```

```

def first_present(mapping, keys):
    for key in keys:
        if isinstance(mapping, dict) and key in mapping and mapping[key] is
not None:
            return mapping[key]
    return np.nan

def extract_usage_from_response(value):
    response = parse_full_api_response(value)
    usage = response.get('usage', {}) if isinstance(response, dict) else {}
    if not isinstance(usage, dict):
        usage = {}

    input_tokens = first_present(usage, ['input_tokens', 'prompt_tokens'])
    output_tokens = first_present(usage, ['output_tokens',
'completion_tokens'])
    total_tokens = first_present(usage, ['total_tokens'])
    cost = first_present(usage, ['cost', 'total_cost', 'estimated_cost'])

    cost_details = usage.get('cost_details', {}) if isinstance(usage, dict)
else {}
    if pd.isna(cost) and isinstance(cost_details, dict):
        cost = first_present(cost_details, ['upstream_inference_cost',
'total_cost', 'cost'])

    return {
        'input_tokens': input_tokens,
        'output_tokens': output_tokens,
        'total_tokens': total_tokens,
        'cost': cost,
    }

def usage_columns_from_results(results, full_api_response_col):
    usage_df = pd.DataFrame(
        list(results[full_api_response_col].apply(extract_usage_from_respon
se)),
        index=results.index,
    )

    fallback_map = {
        'input_tokens': ['input_tokens', 'prompt_tokens'],
        'output_tokens': ['output_tokens', 'completion_tokens'],
        'total_tokens': ['total_tokens'],
        'cost': ['cost'],
    }

    for target_col, candidates in fallback_map.items():
        for fallback_col in candidates:
            source_col = find_col_optional(results, [fallback_col])
            if source_col is not None:
                usage_df[target_col] = usage_df[target_col].fillna(
                    pd.to_numeric(results[source_col], errors='coerce')
                )

```

```

        break

    usage_df['input_tokens'] = pd.to_numeric(usage_df['input_tokens'],
errors='coerce')
    usage_df['output_tokens'] = pd.to_numeric(usage_df['output_tokens'],
errors='coerce')
    usage_df['total_tokens'] = pd.to_numeric(usage_df['total_tokens'],
errors='coerce')
    usage_df['cost'] = pd.to_numeric(usage_df['cost'], errors='coerce')

    missing_total = usage_df['total_tokens'].isna() &
usage_df['input_tokens'].notna() & usage_df['output_tokens'].notna()
    usage_df.loc[missing_total, 'total_tokens'] =
usage_df.loc[missing_total, 'input_tokens'] + usage_df.loc[missing_total,
'output_tokens']

    for token_col in ['input_tokens', 'output_tokens', 'total_tokens']:
        usage_df[token_col] = usage_df[token_col].round().astype('Int64')

    return usage_df[['input_tokens', 'output_tokens', 'total_tokens',
'cost']]

results = normalize_colnames(results)
answers = normalize_colnames(answers)

# Result file aliases
res_n = find_col(results, ['N', 'statement_row_id', 'statement id',
'number', 'problem', 'problem_number'], 'result problem number')
res_datetime = find_col(results, ['datetime', 'date_time', 'timestamp'],
'result datetime')
res_ai_model = find_col(results, ['ai_model', 'model', 'source_file'],
'result AI model')
res_final_numeric = find_col(results, ['final_numeric'], 'result final
numeric')
res_final_formula = find_col(results, ['final_formula'], 'result final
formula')
res_final_units = find_col(results, ['final_units', 'final_unit'], 'result
final units')
res_full_api_response = find_col(results, ['full_api_response',
'api_response', 'response'], 'result full API response')

# Answer-key aliases
ans_n = find_col(answers, ['N', '№', 'No', 'number', 'problem',
'problem_number', 'statement_row_id'], 'answer-key problem number')
ans_actual_numeric = find_col(answers, ['actual_numeric', 'Ans number',
'Ans Number', 'answer_numeric', 'final_numeric', 'numeric'], 'answer-key
numeric answer')
ans_actual_formula = find_col(answers, ['actual_formula', 'Ans LaTeX',
'answer_formula', 'final_formula', 'formula'], 'answer-key formula
answer')

```

```

ans_actual_units = find_col(answers, ['actual_units', 'units', 'Units',
'answer_units', 'final_units', 'final_unit'], 'answer-key units answer')

# Optional formula fallback for rows where `Ans formula` is blank but `Ans
LaTeX` exists.
answer_lower_cols = {str(c).lower().strip(): c for c in answers.columns}
ans_latex_col = answer_lower_cols.get('ans latex')

print('Column mapping:')
print({
    'result_N': res_n,
    'result_datetime': res_datetime,
    'result_ai_model': res_ai_model,
    'result_final_numeric': res_final_numeric,
    'result_final_formula': res_final_formula,
    'result_final_units': res_final_units,
    'result_full_api_response': res_full_api_response,
    'answer_N': ans_n,
    'answer_actual_numeric': ans_actual_numeric,
    'answer_actual_formula': ans_actual_formula,
    'answer_actual_units': ans_actual_units,
    'answer_formula_fallback': ans_latex_col,
})

usage_df = usage_columns_from_results(results, res_full_api_response)

results_clean = results[[res_n, res_datetime, res_ai_model,
res_final_numeric, res_final_formula, res_final_units]].copy()
results_clean.columns = ['N', 'datetime', 'ai_model', 'final_numeric',
'final_formula', 'final_units']
for col in ['input_tokens', 'output_tokens', 'total_tokens', 'cost']:
    results_clean[col] = usage_df[col].values

answers_clean = answers[[ans_n, ans_actual_numeric, ans_actual_formula,
ans_actual_units]].copy()
answers_clean.columns = ['N', 'actual_numeric', 'actual_formula',
'actual_units']

if ans_latex_col is not None:
    blank_formula = answers_clean['actual_formula'].isna() |
(answers_clean['actual_formula'].astype(str).str.strip() == '')
    answers_clean.loc[blank_formula, 'actual_formula'] =
answers.loc[blank_formula, ans_latex_col]

results_clean['N'] = pd.to_numeric(results_clean['N'],
errors='coerce').astype('Int64')
answers_clean['N'] = pd.to_numeric(answers_clean['N'],
errors='coerce').astype('Int64')

merged = results_clean.merge(answers_clean, on='N', how='left',
indicator=True)

unmatched = merged['_merge'].ne('both')

```

```

if unmatched.any():
    print('Warning: rows with no matching answer-key problem number:',
int(unmatched.sum()))

missing_usage = merged[['input_tokens', 'output_tokens', 'total_tokens',
'cost']].isna().any(axis=1)
if missing_usage.any():
    print('Warning: rows with at least one missing usage/cost value:',
int(missing_usage.sum()))

merged = merged.drop(columns=['_merge'])
display(merged.head())

merged['final_numeric_2dp'] =
merged['final_numeric'].apply(numeric_to_2dp)
merged['actual_numeric_2dp'] =
merged['actual_numeric'].apply(numeric_to_2dp)

merged['final_formula_clean'] =
merged['final_formula'].apply(clean_formula)
merged['actual_formula_clean'] =
merged['actual_formula'].apply(clean_formula)

# Output values requested by user:
# - numeric columns rounded to 2 decimals
# - formula columns cleaned of leading assignments like x = ...
# - score columns removed
# - token/cost columns extracted from full_api_response usage data
final_df = pd.DataFrame({
    'N': merged['N'],
    'datetime': merged['datetime'],
    'ai_model': merged['ai_model'],
    'final_numeric': merged['final_numeric_2dp'],
    'final_formula': merged['final_formula_clean'],
    'final_units': merged['final_units'],
    'actual_numeric': merged['actual_numeric_2dp'],
    'actual_formula': merged['actual_formula_clean'],
    'actual_units': merged['actual_units'],
    'input_tokens': merged['input_tokens'],
    'output_tokens': merged['output_tokens'],
    'total_tokens': merged['total_tokens'],
    'cost': merged['cost'],
})

display(final_df.head())

final_df.to_csv(OUTPUT_CSV, index=False)
print(f'Saved: {OUTPUT_CSV}')

```

```
print('\nUsage totals:')
print(final_df[['input_tokens', 'output_tokens', 'total_tokens',
'cost']].sum(numeric_only=True))

if files is not None:
    files.download(OUTPUT_CSV)
else:
    print(f'Output is available at: {Path(OUTPUT_CSV).resolve()}')
```